ProofChain

Flutter SDK Developer guide

Initialize securely. Integrate real workflows. Use the complete Dart API.

A practical implementation manual for Android/iOS Flutter applications and trusted native Dart processes. Includes runnable Dart examples, authenticated Flutter integration, native event streams, ingestion and a source-derived API reference.

Package `proofchain_sdk` · **Version** 0.1.1 · **Edition** 2026-10-15

Reference Current local JavaScript SDK and Dart implementation

Scope 24 resource namespaces · 272 matching public operations

Native Dart. Android/iOS build and device validation remain outstanding. Registry releases are versioned separately from this source guide.

Contents

Section titles and resource namespaces are clickable in the PDF. Code examples use fixture identifiers and require your own authorized environment; no production credentials are included.

1. Scope, status, and platform boundaries

2. Install the package and configure networking

3. Choose routing, credentials, and client lifetime

4. Read data and integrate an authenticated Flutter screen

5. Model, JSON, and error semantics

6. Users, profiles, and pagination

7. Documents, binary files, certificates, and event evidence

8. Events, search, history, and channels

9. Standalone ingestion: two distinct modes

10. Quests, rewards, credentials, and NFT REST

11. Cohorts, Fanpass, DataViews, wallets, and passports

12. Native notifications and foreground lifecycle

13. Production integration and security

14. JavaScript migration and troubleshooting

15. Development, verification and release checklist

16. API reference

client.documents

client.events

client.channels

client.certificates

client.webhooks

client.vault

client.search

client.verifyResource

client.tenant

client.passports

client.wallets

client.users

client.rewards

client.quests

client.schemas

client.dataViews

client.cohorts

client.fanpassLeaderboard

client.notifications

client.credentials

client.nft

client.partnerKeys

client.attestations

client.airkit

Standalone ingestion clients

17. Model and type catalog

1. Scope, status, and platform boundaries

This guide describes source version `0.1.1` of the native Dart package `proofchain_sdk`. Its canonical public entrypoint is `package:proofchain_sdk/proofchain_sdk.dart`; the existing `proofchain.dart` entrypoint exports the same API. The package mirrors the JavaScript SDK's API surface and wire behavior; it is not a JavaScript bridge and does not run Node.js.

1.1 What you can build

Use the package from Flutter on Android/iOS or from a native Dart VM application. Its transport uses `dart:io`; **Flutter Web and browser Dart are not supported**. The package requires Dart `>=3.4.0 <4.0.0` and has no runtime package dependencies. A Flutter app still depends on the Flutter SDK normally.

The main client exposes REST resources, including users, events, quests, rewards, passports, credentials, and wallet REST operations. Native server-sent events (SSE) are available through the notification client. SSE is an application connection, not FCM/APNs push delivery, and does not imply reliable background execution on a phone. The separate embedded `wallet-sdk` is outside this package and this guide. `ProofChain.wallets` does not establish an embedded signing UI or local private-key custody.

1.2 Evidence and limits

Before this documentation task, the implementation was reported as passing analysis under Dart 3.13.5, 17 behavior tests, and 583 runtime parity scenarios against the local JS SDK. Those are prior implementation results, not checks run while writing these chapters. Android/iOS builds, physical devices, and platform lifecycle behavior have **not** been verified by that evidence. A successful parity case also does not establish that a particular production JWT may call an endpoint: the server still decides authorization.

Examples use replaceable synthetic fixtures such as tenant slug `acme`, tenant identifier `tenant-demo`, user identifier `user-demo`, and quest identifier `quest-demo`. These are not provisioned accounts. Obtain real identifiers and an appropriately scoped JWT through your existing authenticated application flow. Never copy a privileged API key into a mobile application to make an example succeed.

2. Install the package and configure networking

2.1 Use a published release or a reviewed local checkout

The package is available on pub.dev. In your app's `pubspec.yaml`, merge this dependency into the existing `dependencies` section:

```
dependencies:
  flutter:
    sdk: flutter
  proofchain_sdk: ^0.1.0
```

The constraint accepts compatible published 0.1.x releases. A GitHub push does not publish a new registry version. To use source changes that have not yet been released, clone the public Flutter SDK repository beside your app instead:

```
workspace/
  mobile_app/
    pubspec.yaml
    lib/
  proofchain-flutter-sdk/
    pubspec.yaml
    lib/proofchain_sdk.dart
```

Replace the registry dependency with a path dependency; do not declare both:

```
dependencies:
  flutter:
    sdk: flutter
  proofchain_sdk:
    path: ../proofchain-flutter-sdk
```

The path is relative to the app's `pubspec.yaml`, not the shell's working directory. From `workspace/mobile_app`, run:

```
flutter pub get
```

For a native Dart application, omit the Flutter SDK dependency and run `dart pub get` from that application's root. Import a public entrypoint rather than `lib/src` files. Path dependencies reflect the local checkout, so pin and review the source used by your team and CI.

2.2 Android: grant Internet access in the main manifest

In the app's `android/app/src/main/AndroidManifest.xml`, add the Internet permission as a direct child of the existing `manifest` element, outside `application`. A permission present only in a debug manifest does not establish release-app access.

```
<uses-permission android:name="android.permission.INTERNET" />
```

This is a manifest declaration, not a runtime permission prompt. Use HTTPS and normal certificate verification. Do not broadly enable cleartext traffic to work around a deployment or certificate problem.

2.3 iOS: keep HTTPS and transport security intact

The default service URLs use HTTPS. Use a trusted HTTPS endpoint for custom `baseUrl` values as well. Do not add `NSAllowsArbitraryLoads`, disable certificate verification, or add broad App Transport Security exceptions for this SDK. The package does not configure your app's `Info.plist`, signing, deployment target, or network entitlements. Check device reachability and certificate trust in your app's environment; a desktop Dart request is not an iOS deployment test.

3. Choose routing, credentials, and client lifetime

3.1 Endpoint precedence

`ProofChain(ProofChainOptions(...))` selects its base URL in this order:

1. A nonempty `baseUrl`, used explicitly.
2. Otherwise a nonempty `tenant` slug, expanded by `tenantApiUrl(tenant)` to `https://<slug>.proofchain.co.za/api`.
3. Otherwise `https://api.proofchain.co.za`.

One trailing slash is removed. The SDK appends resource paths to that base; it does not automatically add `/api` to an explicit URL. For example, an explicit `https://acme.proofchain.co.za/api` keeps the `/api` path. Configure the exact base agreed with your service operator.

`tenant` is a hostname-routing slug. `tenantId` is the tenant identity sent with an end-user bearer token; they are not interchangeable. Setting `tenantId` alone does not change the base URL. `forEndUser` accepts `tenantId` and an optional explicit `baseUrl`, but no `tenant` slug argument.

`ProofChainOptions` field	Type	Default / behavior
`apiKey`	`String?`	Unset; nonempty value sends `X-API-Key`. Trusted backend only.
`userToken`	`String?`	Unset; nonempty value sends `Authorization: Bearer …`.
`tenantId`	`String?`	Unset; sends `X-Tenant-ID` only when both token and tenant ID are nonempty.
`baseUrl`	`String?`	Explicit endpoint; wins over `tenant`. Empty string falls through.
`tenant`	`String?`	Optional routing slug; wins over the global default.
`timeout`	`num?`	Milliseconds; omitted or zero uses `30000`, converted to an integer.
`maxRetries`	`num?`	`3`; `0` disables transport network-error retries.

Use sensible positive integer timeout values and nonnegative integer retry counts even though the signatures use `num`. Timeout covers opening a connection, sending the request, redirects, and obtaining response headers; it is **not** a whole-response-body or full-operation deadline. Retry delays add elapsed time.

The shared HTTP transport retries network failures, not HTTP status responses. It does not automatically retry 429, 401, 403, or 5xx responses, and its typed `TimeoutError` is not retried by that network-failure loop. The default permits up to three retries after the initial transport attempt. See chapter 5 before allowing retries around writes.

3.2 Mobile: use an end-user JWT

Your authenticated host supplies the JWT, tenant ID, and user identity. The SDK does not implement login, exchange credentials for tokens, store tokens, refresh tokens, or select a secure-storage package. Token acquisition and persistence remain application/backend responsibilities. Do not log tokens or put them in source code, assets, crash reports, URLs, or production build-time defines.

This standalone library creates a client from already acquired credentials. It makes no network request; its caller owns and must close the returned client. `baseUrl` should come from trusted application configuration, not untrusted UI input. Credentials are sent to that endpoint.

```
import 'package:proofchain_sdk/proofchain.dart';

ProofChain createMobileClient({
  required String jwt,
  required String tenantId,
  String? baseUrl,
}) {
  return ProofChain.forEndUser(
    userToken: jwt,
    tenantId: tenantId,
    baseUrl: baseUrl,
    timeout: 30000,
    maxRetries: 0,
  );
}
```

The JWT is a capability, not proof that every resource is accessible. A particular read or write can still receive 403. This example chooses zero network retries to make transport replay an explicit application decision; that is not an assertion that all reads require zero retries.

Client credentials are fixed at construction. On refresh or account/tenant switch, stop subscriptions using the old credentials, replace the main `ProofChain` client, and restart only the subscriptions the new session may use. `ProofChain.close()` closes its notifications and HTTP transport. Explicitly cancel any app-owned stream listeners/timers as well. Do not keep resource objects from the old client and expect them to pick up a new token. Closing is resource cleanup, not a public per-request cancellation API or a guarantee that a write already accepted by the server was rolled back.

3.3 Backend-only API keys and environment configuration

Use privileged API keys only in a trusted server or administrative Dart VM process. Mobile binaries, obfuscation, and build defines cannot keep such keys secret. Although the generic options allow both auth fields, do not combine a privileged key with an end-user token to bypass mobile authorization.

`ProofChain.fromEnv()` reads exactly `PROOFCHAIN_API_KEY` and `PROOFCHAIN_BASE_URL` from `Platform.environment`. It throws `StateError` if the API key is absent or empty. It does not load `.env` files, read `--dart-define`, or read JWT, tenant ID, tenant slug, timeout, or retry environment variables. Supply those through explicit options when needed. Environment configuration is for the trusted Dart process, not a mobile secret-injection strategy.

This standalone backend program uses the deployment-provided environment. Its `tenantInfo()` call requires authorization. No credentials are embedded:

```
import 'dart:io' show exitCode, stderr;
import 'package:proofchain_sdk/proofchain.dart';

Future<void> main() async {
  ProofChain? client;
  try {
    client = ProofChain.fromEnv();
    await client.tenantInfo();
    print('Authorized tenant information was received.');
  } catch (_) {
    stderr.writeln('Tenant lookup failed. Check configuration and credentials.');
    exitCode = 1;
  } finally {
    client?.close();
  }
}
```

4. Read data and integrate an authenticated Flutter screen

4.1 First executable API read

This complete native Dart program takes an ephemeral local-demo JWT, tenant ID, and known external user ID as arguments. It reads `users.get(externalId)`; `getByInternalId` is a different method. Run it only with a disposable demo token in a controlled local environment: command arguments can appear in process listings or shell history and are not a production credential channel. The program deliberately does not print the user payload or token.

Save as `bin/read_user.dart` in a Dart/Flutter project with the path dependency. Invoke with three actual demo arguments using `dart run bin/read_user.dart`. There are no bundled credentials, fake responses, or authentication endpoints. The token must be authorized to read the supplied external user.

```
import 'dart:io' show exitCode, stderr;
import 'package:proofchain_sdk/proofchain.dart';

Future<void> main(List<String> args) async {
  if (args.length != 3) {
    stderr.writeln('Expected a local-demo JWT, tenant ID, and external user ID.');
    exitCode = 64;
    return;
  }
  final client = ProofChain.forEndUser(
    userToken: args[0],
    tenantId: args[1],
    maxRetries: 0,
  );
  try {
    final user = await client.users.get(args[2]);
    print('User read completed; status: ${user.status}');
  } catch (_) {
    stderr.writeln('User read failed. Check the demo session and permissions.');
    exitCode = 1;
  } finally {
    client.close();
  }
}
```

Creation occurs before `try` because no owned client exists if construction fails. Once construction succeeds, `finally` runs for both successful reads and exceptions. In a long-lived app, retain a client for the authenticated session rather than constructing and closing one in every widget build.

4.2 Flutter integration: a complete authenticated screen library

The following is Flutter-only code. Save it as `lib/proofchain_app.dart` in a Flutter app with the Flutter SDK and `proofchain_sdk` path dependencies. It is a complete library, not a replacement authentication implementation.

`buildProofChainApp` is the entry into an **existing authenticated bootstrap**: call it instead of that bootstrap's existing `runApp` once it has a real JWT, tenant ID, and user ID. It calls `runApp` itself; do not nest it inside another widget builder. It intentionally has no invented `main` or login API. A host that already owns the top-level `MaterialApp` can instead mount `ProofChainEventsScreen` under that app, supplying the same constructor values.

Here `userId` is the user identifier expected by the events API, not automatically a `users.get` external ID. Use the correct identity from your host's authorized session/domain mapping. This requests the first 20 matching events, not a complete paginated history. The server must permit the JWT to list those events.

```
import 'package:flutter/material.dart';
import 'package:proofchain_sdk/proofchain.dart' as pc;

void buildProofChainApp({
  required String jwt,
  required String tenantId,
  required String userId,
  String? baseUrl,
}) {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(MaterialApp(
    title: 'ProofChain activity',
    home: ProofChainEventsScreen(
      jwt: jwt,
      tenantId: tenantId,
      userId: userId,
      baseUrl: baseUrl,
    ),
  ));
}

class ProofChainEventsScreen extends StatefulWidget {
  const ProofChainEventsScreen({
    super.key,
    required this.jwt,
    required this.tenantId,
    required this.userId,
    this.baseUrl,
  });

  final String jwt;
  final String tenantId;
  final String userId;
  final String? baseUrl;

  @override
  State<ProofChainEventsScreen> createState() => _EventsState();
}

class _EventsState extends State<ProofChainEventsScreen> {
  late pc.ProofChain _client;
  late Future<List<pc.Event>> _events;

  @override
  void initState() {
    super.initState();
    _openSession();
```

```
}

void _openSession() {
  _client = pc.ProofChain.forEndUser(
    userToken: widget.jwt,
    tenantId: widget.tenantId,
    baseUrl: widget.baseUrl,
    maxRetries: 0,
  );
  _events = _readEvents();
}

Future<List<pc.Event>> _readEvents() {
  return _client.events.list(pc.ListEventsRequest(
    userId: widget.userId,
    limit: 20,
  ));
}

@override
void didUpdateWidget(covariant ProofChainEventsScreen oldWidget) {
  super.didUpdateWidget(oldWidget);
  if (oldWidget.jwt != widget.jwt ||
      oldWidget.tenantId != widget.tenantId ||
      oldWidget.userId != widget.userId ||
      oldWidget.baseUrl != widget.baseUrl) {
    _client.close();
    _openSession();
  }
}

void _reload() {
  if (!mounted) return;
  setState(() {
    _events = _readEvents();
  });
}

@override
void dispose() {
  _client.close();
  super.dispose();
}

String _errorMessage(Object? error) {
  if (error is pc.AuthenticationError) {
    return 'Your session needs attention. Return to sign-in.';
  }
  if (error is pc.AuthorizationError) {
    return 'This session cannot view these events.';
  }
  if (error is pc.RateLimitError) {
    return 'Too many requests. Wait before trying again.';
  }
  if (error is pc.NetworkError || error is pc.TimeoutError) {
    return 'Could not reach the service. Check your connection.';
  }
  return 'Events could not be loaded.';
}
```

```
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Recent activity')),
      body: FutureBuilder<List<pc.Event>>(
        future: _events,
        builder: (context, snapshot) {
          if (snapshot.connectionState != ConnectionState.done) {
            return const Center(child: CircularProgressIndicator());
          }
          if (snapshot.hasError) {
            return Center(
              child: Column(
                mainAxisSize: MainAxisSize.min,
                children: [
                  Text(_errorMessage(snapshot.error)),
                  TextButton(
                    onPressed: _reload,
                    child: const Text('Try read again'),
                  ),
                ],
              ),
            );
          }
          final events = snapshot.data ?? const <pc.Event>[];
          if (events.isEmpty) {
            return const Center(child: Text('No recent events.'));
          }
          return ListView.builder(
            itemCount: events.length,
            itemBuilder: (context, index) {
              final event = events[index];
              return ListTile(
                title: Text(event.eventType),
                subtitle: Text(event.timestamp),
              );
            },
          );
        },
      ),
    );
  }
}
```

The `Future` is cached in state, never created in `build`; layout rebuilds do not trigger repeated requests. `FutureBuilder` owns async rendering, so there is no manual `setState` after awaiting network work. The retry callback checks `mounted` before updating state. During reload or credential replacement, loading takes precedence over any data retained by `FutureBuilder`, preventing old-session data from being rendered while the new future is pending.

The screen constructs and owns its client, so it closes it in `dispose` and replaces it in `didUpdateWidget`. A host refreshes credentials by rebuilding with new constructor values; it does not mutate the old client. This screen has no subscriptions. If adding SSE later, stop old listeners before opening the replacement

session and cancel app-owned subscriptions in `dispose`. If you instead inject a shared client, its session owner—not every child widget—must close it. Never close a host-owned shared client merely because one route exits.

5. Model, JSON, and error semantics

5.1 Typed constructors do not normalize every field name

Named TypeScript interfaces become Dart classes with named constructor arguments, getters, `fromJson(JsonMap)`, and `toJson()`. Anonymous object and `Partial<...>` inputs generally remain `JsonMap`, which is `Map<String, Object?>`. Use the signature of the actual method rather than translating JS syntax mechanically. Optional positional method arguments remain positional: `events.list(request)`, not `events.list(request: request)`.

Field casing follows the source interface, not a universal Dart style rule. For example, `ListEventsRequest.userId` is camelCase, while `UpdateEndUserRequest.display_name` is snake_case. Resource methods can rename fields when preparing wire requests: `events.list` maps `userId` to `user_id`. Do not apply a global casing transform to models or manually rename typed constructor arguments.

Reserved source names are mapped where necessary. `SchemaField.defaultValue` is the Dart property for the JSON key `default`. `SchemaField.required` is actually named `required`; do not invent a `required_` property. This standalone serialization example performs no network operation:

```
import 'package:proofchain_sdk/proofchain.dart';

JsonMap schemaFieldJson() {
  const field = SchemaField(
    name: 'attendance',
    type: 'string',
    required: true,
    defaultValue: 'pending',
  );
  final decoded = SchemaField.fromJson(field.toJson());
  return decoded.toJson();
}
```

5.2 Omitted and explicitly null are different

For the models illustrated here, normal constructors omit optional null fields from `toJson`. Passing `email: null` to `UpdateEndUserRequest` therefore does **not** itself express an explicit JSON null. Models decoded with `fromJson` retain the original map for `toJson`, preserving explicit nulls and extra keys. For example:

```
import 'package:proofchain_sdk/proofchain.dart';

Map<String, JsonMap> compareUpdatePayloads() {
  const omitted = UpdateEndUserRequest(email: null);
  final explicit = UpdateEndUserRequest.fromJson(
    <String, Object?>{'email': null},
  );
  return {
    'omitted': omitted.toJson(),
    'explicit': explicit.toJson(),
  };
}
```

The returned `omitted` payload has no `email` key; `explicit` contains that key with a null value. This is a serialization distinction, not a promise that the backend allows clearing an email. Field-level null/clear semantics still belong to the endpoint contract and server validation.

Do not use `fromJson` as a schema validator. Incorrect field types can cause cast errors during decoding; missing required fields can cause errors when required getters are accessed. Treat the input map as immutable after decoding: retained original maps are not deep-copied, and mutating them is not a reliable way to update already decoded getters. To derive a request, construct a new model or a new map. `toJson` output may contain personal data; do not log entire responses by default.

5.3 Partial updates use explicit maps

`quests.update(String questId, JsonMap data)` accepts a partial object and uses HTTP PUT internally. Do not infer its signature from the HTTP verb, invent an `UpdateQuestRequest`, or send a whole fetched model when only one field should change. The following complete library accepts a configured client; its caller retains ownership. This is an authorized write, normally a trusted administrative operation. `quest-demo` is a replaceable fixture, not a known server record.

```
import 'package:proofchain_sdk/proofchain.dart';

Future<Quest> renameQuest(ProofChain client, String questId) {
  final JsonMap changes = {'name': 'Acme welcome quest'};
  return client.quests.update(questId, changes);
}
```

Map keys are backend/source field names. Omit a key to leave it out of the request; include a null only when the server documents that null behavior. Keep nested values JSON-compatible: strings, booleans, finite numbers, nulls, lists, maps with string keys, or supported SDK models. Do not pass arbitrary `DateTime`, `File`, or custom application objects to a JSON request. Timestamps in models are strings; parse them explicitly in your presentation layer when needed. Types expressed as `num` are not automatically integer-only validators.

5.4 Handle typed failures without leaking raw payloads

Catch specific errors before `ProofChainError`. Exceptions are not returned as success models. The shared HTTP status mapping is:

Failure	SDK exception	Application response
401	`AuthenticationError`	Ask the host session manager to reauthenticate or refresh.
403	`AuthorizationError`	Explain access denial; do not retry with a privileged mobile key.
404	`NotFoundError`	Handle a missing record in the current authorized context.
400 or 422	`ValidationError`	Inspect safely shaped details; correct the request.
429	`RateLimitError`	Respect a usable delay and your application retry policy.
5xx	`ServerError`	Surface a service failure; reconcile writes before resubmission.
Other non-success status	`ProofChainError`	Inspect `statusCode` without assuming a known subtype.
Transport failure after network retries	`NetworkError`	Distinguish uncertain write outcome from a confirmed rejection.
Connection/header deadline exceeded	`TimeoutError`	Handle timeout; it is not evidence that a write did not run.

This table describes the main client's shared HTTP transport, not every exported helper. Standalone ingestion clients have their own timeout and lifecycle behavior; do not assume all SDK timeouts become `TimeoutError`.

`ProofChainError` exposes `message`, nullable `statusCode`, and nullable `responseBody`; do not assume all subclasses retain a response body. Even an error message may contain server-controlled or sensitive content. Display curated UI messages and use redacted diagnostics.

`ValidationError.rawErrors` is `Object?`: the server can send a map, string, list, null, or other shape. Its lazy `errors` getter expects a list of `ValidationErrorDetail`-shaped entries and may throw for nonstandard data. Do not blindly evaluate it inside a catch block. `RateLimitError.retryAfter` is nullable `num` in **seconds**, not a `Duration` or milliseconds. Header parsing can produce a nonfinite number for unusable input; it does not implement full HTTP-date scheduling. Check nullability, finiteness, and sign before using it.

This standalone library produces safe, bounded summaries for an application's error boundary. It handles arbitrary raw validation shapes without echoing server strings. Its return value is a message, not a retry action.

```
import 'package:proofchain_sdk/proofchain.dart';

int countFieldErrors(Object? raw) {
  if (raw is! List) return 0;
  var count = 0;
```

```
  for (final item in raw) {
    if (item is ValidationErrorDetail) {
      count++;
    } else if (item is Map &&
        item['field'] is String &&
        item['message'] is String) {
      count++;
    }
  }
  return count;
}

String describeFailure(Object error) {
  if (error is AuthenticationError) return 'Sign-in needs attention.';
  if (error is AuthorizationError) return 'Access is not permitted.';
  if (error is NotFoundError) return 'The requested record was not found.';
  if (error is ValidationError) {
    final count = countFieldErrors(error.rawErrors);
    return count == 0
        ? 'The request was rejected. Check the submitted values.'
        : 'The request has $count field validation issue(s).';
  }
  if (error is RateLimitError) {
    final seconds = error.retryAfter;
    if (seconds != null && seconds.isFinite && seconds >= 0) {
      return 'Rate limited; suggested wait: $seconds seconds.';
    }
    return 'Rate limited; no usable retry delay was supplied.';
  }
  if (error is TimeoutError) return 'The request timed out.';
  if (error is NetworkError) return 'A network failure occurred.';
  if (error is ServerError) return 'The service could not complete the request.';
  if (error is ProofChainError) return 'The API request failed.';
  return 'An unexpected application or response-format error occurred.';
}
```

A catch-all boundary is still needed for malformed successful JSON, model cast errors, and application failures, which are not necessarily wrapped in `ProofChainError`. Do not relabel every unexpected exception as a network error.

5.5 Avoid blind write retries

A connection can fail after the server has applied a write but before the app receives its response. The shared network retry loop is not restricted to GET; with its default retry setting, transport failures can replay writes as well. There is no general SDK idempotency-key facility or automatic write reconciliation. A timeout, socket failure, or 5xx therefore must not trigger an unconditional application-level retry of operations such as points awards, claims, wallet creation, settlement, or event creation.

For sensitive workflows, configure `maxRetries: 0` when constructing the client and use an endpoint-specific reconciliation/confirmation policy supplied by your backend. Do not silently add another retry loop around a helper that already performs multiple requests. For reads, offer an explicit bounded retry or user retry after considering authorization, rate limits, and the actual failure. The Flutter screen's button retries only its event **read**; it is not a pattern for replaying arbitrary mutations.

6. Users, profiles, and pagination

These recipes target native Android/iOS and the Dart VM, not Flutter Web. Each Dart block is an independent library importing `package:proofchain_sdk/proofchain.dart`. Add the SDK dependency before using them. Functions accept an already configured `ProofChain` and known identifiers; the caller owns and eventually closes that client. Functions do not run merely by being imported.

Values such as `acme`, `user-demo`, `quest-demo`, `lesson_completed`, and sample metadata are replaceable fixtures, not pre-existing records or a backend protocol. Use identifiers returned by your deployment. Every request is subject to server authorization and tenant isolation. A JWT identifies an end user; it does not grant access to administrative routes. Put privileged operations in a trusted backend or explicitly authorized native administration process, never ship a privileged API key inside a mobile application.

6.1 External IDs and editable profile data

`users.get`, `update`, `setProfile`, and attribute helpers address the user's external ID. Methods explicitly named `getByInternalId` and `updateByInternalId` address the internal ID. `delete`, GDPR preview, and GDPR deletion use the internal ID. Do not exchange these identifiers just because both are strings.

A profile editor should submit only permitted fields and render the returned user, rather than assuming that a local edit was accepted. This example changes a display name and application metadata; it requires the corresponding server-authorized user scope.

```
import 'package:proofchain_sdk/proofchain.dart';

Future<EndUser> saveProfile(
  ProofChain client,
  String externalId,
  String displayName,
) async {
  await client.users.setProfile(
    externalId,
    SetProfileRequest(display_name: displayName, language: 'en'),
  );
  return client.users.updateAttributes(externalId, {
    'preferred_topic': 'developer-learning',
    'onboarding_version': 1,
  });
}
```

The SDK sends `{'attributes': attributes}` for `updateAttributes`; it does not merge the supplied map with a locally fetched copy. Confirm your server's patch semantics before relying on preservation of unrelated keys. `removeAttributes` reads the existing attributes, removes keys, then writes the resulting map. That read-modify-write is not atomic: concurrent editors can overwrite changes. Keep secrets and sensitive personal information out of unneeded metadata.

6.2 Backend/admin: provision users and enumerate pages

Provisioning and whole-tenant enumeration belong in authorized administration. Creation is not an upsert and the SDK does not invent a duplicate-resolution policy. An external ID should be stable within your own identity system.

```
import 'package:proofchain_sdk/proofchain.dart';

Future<EndUser> provisionUser(
  ProofChain admin,
  String externalId,
) {
  return admin.users.create(CreateEndUserRequest(
    external_id: externalId,
    display_name: 'Acme learner',
    attributes: {'source': 'acme-enrolment'},
  ));
}

Stream<EndUser> enumerateUsers(ProofChain admin) async* {
  num page = 1;
  while (true) {
    final result = await admin.users.list(
      ListEndUsersOptions(page: page, page_size: 50),
    );
    for (final user in result.users) {
      yield user;
    }
    if (!result.has_more) break;
    page = result.page + 1;
  }
}
```

`EndUserListResponse` exposes `users`, `total`, `page`, `page_size`, and `has_more`. This is page-based pagination, unlike APIs using `limit`/`offset`. The sample size of 50 is an application choice, not a documented server maximum. For large directories, consume the stream incrementally instead of retaining all users. Directory changes during traversal can affect page membership; the SDK does not provide a snapshot cursor.

Filters include `search`, `status`, `segment`, `has_wallet`, `min_events`, `sort_by`, and `sort_order`. Boolean `false` and numeric zero are retained for `has_wallet` and `min_events`. Avoid assuming every other API treats falsy options identically. `users.export` returns CSV/JSON bytes, not a browser download. Its options map accepts `segment` and `format`; the caller must choose where to save bytes and how to protect exported personal information.

Adding points, merging identities, linking/provisioning wallets, and GDPR operations need explicit policy and authorization. None is invoked by these recipes. In particular, do not trigger deletion while merely loading a profile.

7. Documents, binary files, certificates, and event evidence

7.1 Attest a selected document

`UploadFile` is an in-memory value. Obtain bytes using your app's picker or a caller-supplied native file path; the SDK does not request picker permissions. The following library additionally depends on Dart's `dart:io`. It reads only the supplied path and does not assume any bundled fixture file exists.

```
import 'dart:io';
import 'package:proofchain_sdk/proofchain.dart';

Future<AttestationResult> attestSelectedDocument(
  ProofChain client,
  String userId,
  String selectedPath,
  String filename,
) async {
  final bytes = await File(selectedPath).readAsBytes();
  return client.documents.attest(AttestRequest(
    file: UploadFile(bytes, filename: filename),
    filename: filename,
    userId: userId,
    eventType: 'document_uploaded',
    metadata: {'application': 'acme-learning'},
    encrypt: true,
  ));
}
```

This is a multipart request, not a JSON-encoded file. Metadata is serialized as JSON within a multipart field. `encrypt: true` sends `encrypt=1`; omitted or false does not send that flag. Encryption is a server request, not client-side cryptography or a promise about key ownership.

Set a nonempty filename explicitly in **both** the upload value and request. `UploadFile` defaults to `document`, but `documents.attest` chooses the request's `filename` or `document`, and vault upload chooses the request's filename or `file`. NFT minting chooses `imageFilename` or `image.png`. An `UploadFile.filename` alone therefore is not a universal filename override. Use `contentType` when known; its default is `application/octet-stream`.

`AttestationResult` contains `id`, `ipfsHash`, `documentHash`, `gatewayUrl`, `verifyUrl`, `certificateId`, `status`, `attestationMode`, and `timestamp`, with optional `blockchainTx` and `proof`. Preserve these distinctions: a document hash, an IPFS hash, an event ID, and a certificate ID are not interchangeable. A returned record or queued status does not by itself mean blockchain finality. `documents.get(ipfsHash)` returns an `Event`, not document bytes.

7.2 Vault storage and authenticated binary retrieval

Vault storage is a separate resource from document attestation. Uploading to the vault does not call `documents.attest`. The next recipe returns downloaded bytes to its caller rather than writing an arbitrary destination or opening a URL. It needs `dart:typed_data` in addition to the SDK dependency.

```
import 'dart:typed_data';
import 'package:proofchain_sdk/proofchain.dart';

Future<Uint8List> storeAndReadVaultFile(
  ProofChain client,
  String userId,
  Uint8List bytes,
  String filename,
) async {
  final stored = await client.vault.upload(VaultUploadRequest(
    file: UploadFile(bytes, filename: filename),
    filename: filename,
    userId: userId,
    accessMode: 'private',
  ));
  return client.vault.download(stored.id);
}
```

`vault.download` returns `Uint8List` through the configured authenticated HTTP client. It is not the `gatewayUrl` returned by attestation. Both upload and binary retrieval buffer data in memory; set application size limits before reading a large selected file. No SDK upload-size maximum is enforced here. Folders, moves, and expiring shares are server resources; `share(fileId, expiresInHours)` uses hours, not milliseconds. Share response data is `JsonMap`: inspect the actual server contract rather than assuming an undocumented URL key. Vault deletions are explicit operations and are not part of this recipe.

7.3 Consume certificate verification without inventing a boolean

Use the typed verification resource when you want the modeled response. `certificates.verify` returns a raw `JsonMap`; `verifyResource.certificate` returns `CertificateVerifyResult`. The typed result has `status`, not a `valid` boolean. Present the actual status, expiration, and revocation information using your product's verification policy.

```
import 'package:proofchain_sdk/proofchain.dart';

Future<CertificateVerifyResult> inspectCertificate(
  ProofChain client,
  String certificateId,
  void Function(String status, String? revokedAt) present,
) async {
  final result = await client.verifyResource.certificate(certificateId);
  present(result.status, result.revokedAt);
  return result;
}
```

The response also includes recipient/title, issue and optional expiry times, IPFS hash, verification URL, and optional blockchain metadata. A network failure is not a negative verification decision. Keep “could not check” distinct from a server-reported revoked or otherwise unacceptable certificate.

For other evidence, `client.verify(ipfsHash)` is the main hash verification route; `verifyResource.event(ipfsHash)` returns raw event verification data. `eventBatchProof(eventId)` retrieves a batch proof; `proof(leaf, proof, root)` submits explicit proof material. `batch(batchId)` and `batchVerify(items)` are different calls with different identifiers/payloads. The SDK does not locally prove authenticity merely by deserializing these responses.

7.4 Backend/admin: issue a certificate

Certificate issuance is a privileged business operation. Keep recipient policy, consent, and allowed metadata on the trusted side. It is separate from obtaining a certificate ID as part of event ingestion.

```
import 'package:proofchain_sdk/proofchain.dart';

Future<Certificate> issueCourseCertificate(
  ProofChain admin,
  String recipientName,
  String recipientEmail,
) {
  return admin.certificates.issue(IssueCertificateRequest(
    recipientName: recipientName,
    recipientEmail: recipientEmail,
    title: 'Acme developer course',
    metadata: {'course': 'developer-basics'},
  ));
}
```

Listing certificates uses `ListCertificatesRequest` with optional recipient email and `limit`/`offset`. Revocation is an explicit authorized action, `revoke(certificateId, reason)`, not a side effect of verification.

8. Events, search, history, and channels

8.1 Write an authorized event and search its domain

Event writes must be permitted by the server for the current caller. If an ordinary mobile user should submit only end-user ingestion events, use chapter 9 rather than granting tenant event-write privileges. A client-supplied `source` is metadata, not proof of authorization.

```
import 'package:proofchain_sdk/proofchain.dart';

Future<SearchResult> recordAndSearchLesson(
  ProofChain client,
  String userId,
  String lessonId,
) async {
  await client.events.create(CreateEventRequest(
    eventType: 'lesson_completed',
    userId: userId,
    source: 'acme-learning',
    data: {'lesson_id': lessonId},
  ));
  return client.events.search(SearchRequest(
    query: lessonId,
    userId: userId,
    eventType: 'lesson_completed',
    page: 1,
    limit: 20,
  ));
}
```

`SearchResult` exposes `events`, `total`, `page`, and `limit`; render the result set instead of treating search success as confirmation that a just-written event is already indexed. `events.list` returns `List<Event>` and uses offset-based options. `events.createBatch` is a sequential loop over `create`, **not** an atomic batch endpoint: earlier writes can succeed before a later failure. Do not retry the whole list blindly after an ambiguous response.

`search.query` is a separate structured `SearchQueryRequest` API, with filters, offset, limit, and `includeData`. `search.quick` accepts a text query and optional positional limit. Do not copy a `SearchRequest` into `search.query` or substitute one response model for another.

8.2 User history is a bounded query

```
import 'package:proofchain_sdk/proofchain.dart';

Future<UserEventHistoryResponse> loadRecentHistory(
  ProofChain client,
  String userId,
  num offset,
) async {
  return client.search.byUser(
    userId,
    UserEventHistoryOptions(days: 30, limit: 25, offset: offset),
  );
}
```

The result exposes `results`, `total`, `offset`, `limit`, `query_time_ms`, optional `pagination`, and optional `retention_warning`. Surface retention warnings where users might otherwise mistake a partial history for a complete audit trail. Advance offsets according to the returned page and product requirements. The SDK also accepts a legacy numeric limit as the second positional argument, but a typed options object makes the day window explicit. `users.getActivity` is another user-activity endpoint with `page`, `page_size`, and `event_type` in a `JsonMap`; it is not this history response.

8.3 Backend/admin: channels and explicit settlement ownership

Channels group event submission and settlement. Their `stream` method is an HTTP POST; it does not open a WebSocket or return a Dart stream. The following function submits to an existing authorized channel and returns its acknowledgment.

```
import 'package:proofchain_sdk/proofchain.dart';

Future<StreamAck> submitChannelLesson(
  ProofChain admin,
  String channelId,
  String userId,
  String lessonId,
) {
  return admin.channels.stream(channelId, StreamEventRequest(
    eventType: 'lesson_completed',
    userId: userId,
    source: 'acme-learning',
    data: {'lesson_id': lessonId},
  ));
}

Future<Channel> createLearningChannel(ProofChain admin) {
  return admin.channels.create(const CreateChannelRequest(
    name: 'Acme learning events',
    description: 'Authorized course completion events',
  ));
}
```

Use `status(channelId)` to inspect the channel. `list(limit, offset)` has positional defaults 50 and 0. `streamBatch(channelId, events)` posts an `events` envelope and returns `JsonMap`; do not assume it shares standalone ingestion's batch limits or response fields. `settle` and `close` are explicit server mutations, not local socket cleanup. Assign their timing to an authorized backend workflow. These examples intentionally do not settle or close a channel as an automatic consequence of a screen leaving the foreground.

9. Standalone ingestion: two distinct modes

These clients are independent of `ProofChain`: passing a configured main client does not configure them. Their transports, timeouts, endpoints, and wire payloads differ from the shared HTTP resource client.

Property	`IngestionClient`	`EndUserIngestionClient`
Intended authority	Trusted backend/native administration process	Mobile end-user JWT
Required options	`apiKey`	`userToken`, `tenantId`
Optional options	`ingestUrl`, `timeout`	`apiUrl`, `timeout`
Default origin	`https://ingest.proofchain.co.za`	`https://api.proofchain.co.za`
Single route	`/events/ingest`	`/end-user/events/ingest`
Batch route	`/events/ingest/batch`	`/end-user/events/ingest/batch`
Batch body	Raw event array	Object containing `events` array
Local batch maximum	1,000	100
Client-controlled source	`eventSource`, default `sdk`	Ignored; server stamps provenance
Schema IDs	Single request: `X-Schemas` header	Not sent
Automatic retries	None	None

The actual mobile class is **`EndUserIngestionClient`**, not `IngestionUserClient`. Both use `IngestEventRequest` and `BatchIngestRequest` as Dart inputs even though their wire batch envelopes differ.

9.1 Mobile JWT ingestion

Obtain a JWT from your existing authorized identity flow; this SDK does not define a login route. Pass the tenant and API origin belonging to that identity. `apiUrl` is the API origin used before the `/end-user/...` path, not the main client's tenant-prefixed resource URL.

```
import 'package:proofchain_sdk/proofchain.dart';

Future<IngestEventResponse> ingestMobileLesson(
  String userToken,
  String tenantId,
  String apiOrigin,
  String userId,
  String lessonId,
) async {
  final ingestion = EndUserIngestionClient(EndUserIngestionClientOptions(
    userToken: userToken,
    tenantId: tenantId,
    apiUrl: apiOrigin,
    timeout: 30000,
  ));
  return ingestion.ingest(IngestEventRequest(
    userId: userId,
    eventType: 'lesson_completed',
    data: {'lesson_id': lessonId},
    timestamp: DateTime.now().toUtc().toIso8601String(),
  ));
}
```

`setToken(newToken)` updates a retained end-user ingestion client after your identity layer refreshes credentials. It does not refresh tokens itself. `ProofChain` has no equivalent token setter: manage its lifetime separately. Neither standalone ingestion class has a `close()` method. Every operation creates and closes its own `dart:io` transport in a `finally` block, including failure/timeout paths. Do not add a nonexistent cleanup call.

9.2 Backend/admin: direct ingestion with schema selection

The API key in this recipe must come from trusted runtime configuration. Never compile it into a mobile binary, include it in a checked-in example, or return it from an application backend to an untrusted client.

```
import 'package:proofchain_sdk/proofchain.dart';

Future<IngestEventResponse> ingestTrustedLesson(
  String apiKey,
  String ingestOrigin,
  String userId,
  String lessonId,
  List<String> schemaIds,
) async {
  final ingestion = IngestionClient(IngestionClientOptions(
    apiKey: apiKey,
    ingestUrl: ingestOrigin,
    timeout: 30000,
  ));
  return ingestion.ingest(IngestEventRequest(
    userId: userId,
    eventType: 'lesson_completed',
    eventSource: 'acme-backend',
    schemaIds: schemaIds,
    data: {'lesson_id': lessonId},
  ));
}
```

A nonempty `schemaIds` list becomes the comma-separated `X-Schemas` header for **direct single-event ingestion only**. Direct batches ignore per-event schema IDs. End-user ingestion ignores both schema IDs and `eventSource`; adding them does not grant privileged provenance. `hot: true` is sent when requested; `hot: false` is omitted. `timestamp` is an optional caller-supplied string; the SDK does not infer a domain timestamp or validate its semantics.

9.3 Batch accounting and limits

Do not confuse successful HTTP delivery with all events being queued or confirmed. Consume `totalEvents`, `queued`, `failed`, and individual `results`. The normalized per-event fields are `eventId`, `certificateId`, and `status`. Single responses can additionally expose `queuePosition` and `estimatedConfirmation`; those queue fields are not retained in batch results.

```
import 'package:proofchain_sdk/proofchain.dart';

Future<BatchIngestResponse> ingestMobileBatch(
  EndUserIngestionClient ingestion,
  List<IngestEventRequest> events,
  void Function(num queued, num failed) presentCounts,
) async {
  if (events.length > 100) {
    throw ArgumentError.value(events.length, 'events', 'Maximum is 100');
  }
  final result = await ingestion.ingestBatch(
    BatchIngestRequest(events: events),
  );
  presentCounts(result.queued, result.failed);
  return result;
}
```

For direct ingestion, use the same `BatchIngestRequest` with `IngestionClient.ingestBatch`, up to 1,000 events. Oversized batches throw `ValidationError` before sending. The SDK does not enforce a nonempty minimum. Partition larger work in the application and preserve each batch's outcome; do not assume atomicity, automatic rollback, or a backend idempotency key.

`timeout` is in milliseconds and defaults to 30,000 when omitted or falsy. Batch calls use twice the configured timeout. Unlike the shared HTTP client's connection/header deadline, standalone ingestion applies its timeout to the whole operation, including response-body reading, and closes that operation's transport on timeout. `ingest` and `ingestBatch` report timeout as `NetworkError`. Direct `getEventStatus(eventId)` returns the raw wire-keyed `JsonMap`, uses the single timeout, and can throw `dart:async` `TimeoutException` instead. End-user ingestion has no status method.

Neither mode retries automatically, including network errors or HTTP 429. Handle `RateLimitError` and transient failures at an application boundary; an ambiguous write failure can mean the server already accepted the event. Do not infer a universal retry-safe property from the request returning an exception. The shared resource HTTP client retries network errors, not HTTP status responses; that separate behavior does not apply to these clients.

10. Quests, rewards, credentials, and NFT REST

10.1 Load an authorized quest journey

```
import 'package:proofchain_sdk/proofchain.dart';

Future<QuestWithProgress> openQuest(
  ProofChain client,
  String questId,
  String userId,
) async {
  return client.quests.getWithProgress(questId, userId);
}

Future<List<QuestWithProgress>> availableQuests(
  ProofChain client,
  String userId,
) {
  return client.quests.listAvailable(userId, {'category': 'learning'});
}
```

`quests.list(ListQuestsOptions(...))` supports status, category, public/featured flags, limit, and offset. `listAvailable(userId, options)` uses only its category filter. `listWithProgress(userId, options)` delegates to that available-quests route and forwards category only: it does **not** preserve every list filter.

`getWithProgress` fetches the quest and progress concurrently. It suppresses progress-fetch errors to a nullable progress result while still failing if the quest fetch fails. Consequently, absent progress can mean more than “not yet started.” If the distinction matters to your UI, call `getUserProgress` explicitly and handle its errors separately. Use `resolveCtaLink` only with a trusted quest ID/model or server-advertised path; the helper can accept a path, and callers should not turn arbitrary untrusted input into navigation policy.

10.2 Start, complete an eligible step, then claim when allowed

The SDK has `startQuest`, `startStep`, `completeStep`, and `claimReward`; it has no generic `completeQuest` method. Use the step index required by the backend for the loaded quest. Do not substitute a step ID or assume an array position without checking the quest definition and server contract.

```
import 'package:proofchain_sdk/proofchain.dart';

Future<UserQuestProgress> beginQuest(
  ProofChain client,
  String questId,
  String userId,
) {
  return client.quests.startQuest(questId, userId);
}

Future<ClaimRewardResult?> completeEligibleStep(
  ProofChain client,
  String questId,
  String userId,
  num stepIndex,
) async {
  await client.quests.startStep(questId, userId, stepIndex);
  final completed = await client.quests.completeStep(
    questId,
    userId,
    stepIndex,
  );
  if (!completed.can_claim) return null;
  return client.quests.claimReward(questId, userId);
}
```

Call completion only when the real-world action and server policy allow it, not on screen mount. Some quest types depend on ingested events or other criteria: a POST is not a shortcut around those rules. `StepCompletionResult` reports `step_completed`, `quest_completed`, `can_claim`, current/target counts, and points earned. A claim response reports `status`, `points_awarded`, optional `reward_earned_id`, and `claimed_at`; a reward ID is not a confirmed NFT mint. Refresh progress for authoritative status, completion percentage, step progress, and earned reward information. Partial failure between start, completion, and claim is possible; these are separate requests, not a transaction.

10.3 Read rewards and credentials; request an eligible NFT claim

These functions require server-authorized access to the requested user's records. A wallet address must already be validated/selected by the application; this library does not establish wallet ownership by signing a challenge.

```
import 'package:proofchain_sdk/proofchain.dart';

Future<List<RewardUserReward>> loadUserRewards(
  ProofChain client,
  String userId,
) {
  return client.rewards.getUserRewards(userId);
}

Future<UserCredentialsSummary> loadUserCredentials(
  ProofChain client,
  String externalId,
) {
  return client.credentials.getUserCredentials(externalId);
}

Future<ClaimNFTResult> claimEarnedNft(
  ProofChain client,
  String earnedRewardId,
  String walletAddress,
) {
  return client.rewards.claimNFT(earnedRewardId, walletAddress);
}

Future<NFTMint> inspectMint(ProofChain client, String mintId) {
  return client.nft.get(mintId);
}
```

`getCatalog(category)` returns reward definitions, whereas `getUserRewards` returns user reward records and `listEarned` returns earned rewards. Their models are not interchangeable. Claiming, distribution, attestation, anchoring, and verification are separate calls. Inspect server states rather than presenting a mint as complete just because a claim request was accepted.

Credential verification uses `credentials.verify(verificationCode)`; it is separate from certificate verification and wallet attestation verification. Credential opt-in/opt-out are server mutations affecting a user's enrollment, not mobile notification permission prompts. Avoid invoking them automatically while loading the user's credentials.

10.4 Backend/admin: edit definitions, award, issue, and mint

Quest creation/publishing/pausing, definition editing, manual awards, distribution, revocation, credential issuance/type administration, and direct NFT minting belong to trusted authorized administration. JWT construction does not make those actions safe for a mobile client.

Step helpers are read-modify-write operations over the **full** steps array. `addStep` appends using maximum existing order plus one when order is omitted; `updateStep` and `deleteStep` rewrite existing editable fields; `reorderSteps` rewrites the array using only the IDs supplied. Supply every step ID you intend to retain and serialize administration to avoid lost updates.

```
import 'package:proofchain_sdk/proofchain.dart';

Future<QuestStep> addLearningStep(ProofChain admin, String questId) {
  return admin.quests.addStep(questId, const CreateQuestStepRequest(
    name: 'Complete the introduction',
    event_type: 'lesson_completed',
    required_data_fields: ['lesson_id'],
  ));
}

Future<List<EarnedReward>> awardApprovedReward(
  ProofChain admin,
  String definitionId,
  String userId,
) {
  return admin.rewards.award(ManualRewardRequest(
    definition_id: definitionId,
    user_ids: [userId],
    distribute_immediately: false,
    trigger_data: {'review': 'acme-approved'},
  ));
}

Future<IssuedCredential> issueApprovedCredential(
  ProofChain admin,
  String credentialTypeId,
  String userId,
) {
  return admin.credentials.issue(IssueCredentialRequest(
    credential_type_id: credentialTypeId,
    user_id: userId,
    issue_reason: 'Approved Acme course completion',
  ));
}
```

Direct minting is also a REST operation, not on-device signing. The following function accepts actual image bytes and a deployment-supported network; it sets both upload and multipart request filenames explicitly. It needs `dart:typed_data`. Only invoke after trusted recipient and issuance checks.

```
import 'dart:typed_data';
import 'package:proofchain_sdk/proofchain.dart';

Future<NFTMint> mintApprovedCourseNft(
  ProofChain admin,
  String walletAddress,
  String network,
  Uint8List imageBytes,
  String filename,
) {
  return admin.nft.mint(MintNFTRequest(
    recipientAddress: walletAddress,
    name: 'Acme course completion',
    image: UploadFile(
      imageBytes,
      filename: filename,
      contentType: 'image/png',
    ),
    imageFilename: filename,
    network: network,
    metadata: {'course': 'developer-basics'},
  ));
}
```

Match the declared content type to real bytes; change it for non-PNG input. The SDK can alternatively send `imageUrl`. It does not upload an absent image for you, deploy arbitrary contracts, manage gas locally, or provide a wallet signer. `nft.list` supplies recipient/network/status and limit/offset filters for tracking server-managed mint records.

11. Cohorts, Fanpass, DataViews, wallets, and passports

11.1 Query related analytics with explicit identifiers

Cohort leaderboards operate on a cohort definition ID; Fanpass has its own leaderboard and optional aggregation rule ID. Both accept typed string-valued filter maps, country, limit, `top_n`, `fresh`, and `userId` context. There is no single generic `client.leaderboards` resource in this SDK.

```
import 'package:proofchain_sdk/proofchain.dart';

Future<CohortLeaderboardResponse> loadCohortBoard(
  ProofChain client,
  String cohortId,
  String userId,
) {
  return client.cohorts.getLeaderboard(
    cohortId,
    CohortLeaderboardOptions(limit: 20, top_n: 10, userId: userId),
  );
}

Future<FanpassLeaderboardResponse> loadFanpassBoard(
  ProofChain client,
  String userId,
) {
  return client.fanpassLeaderboard.getLeaderboard(
    FanpassLeaderboardOptions(limit: 20, top_n: 10, userId: userId),
  );
}

Future<JsonMap> loadCustomViewData(
  ProofChain client,
  String identifier,
  String viewName,
) async {
  final view = await client.dataViews.execute(identifier, viewName);
  return view.data;
}
```

Use `cohorts.list`/`get` to discover permitted definitions rather than guessing IDs. `getUserBreakdown` and Fanpass `getUserComparison` accept a `JsonMap` with `filters` and `country`. `fanpassLeaderboard.listCohorts` fetches the complete leaderboard and returns its cohorts; `getCohort` searches that list and can return null. Neither is a separate server route for one cohort.

`fresh: true` requests fresh server evaluation; the SDK does not guarantee a particular cache age. Avoid requesting it on every animation or rebuild. Filter names and aggregation rules are deployment-defined; do not assume arbitrary metadata keys are supported leaderboard filters.

DataViews offer catalog discovery, custom view execution, fan profiles by wallet address, and activity summaries with an optional positional day count (default 30). `execute(identifier, viewName)` returns `DataViewExecuteResult` with `view_name`, `display_name`, `identifier`, `identifier_type`, `data`, `computed_at`, and `total_events`. Interpret `data` using the selected view's schema, not an invented universal metrics object. Custom view creation, updates/deletion, and previewing privileged data require backend/admin scope. A wallet address accepted by one view is not automatically a passport user ID.

11.2 Wallet REST reads and passport views

The main SDK includes `WalletClient` REST APIs. The separate embedded `wallet-sdk` is not part of this package. These REST calls do not provide local key custody, challenge signing, transaction signing, WalletConnect, or an embedded wallet UI.

```
import 'package:proofchain_sdk/proofchain.dart';

Future<ComprehensiveWalletInfo> loadWalletDetails(
  ProofChain client,
  String walletId,
) {
  return client.wallets.getInfo(walletId, {
    'includeBalances': true,
    'includeNfts': true,
    'includeActivity': false,
  });
}

Future<TransactionHistory> loadWalletHistory(
  ProofChain client,
  String walletId,
  num offset,
) {
  return client.wallets.getTransactions(walletId, {
    'limit': 25,
    'offset': offset,
  });
}

Future<PassportWithFields> loadPassportFields(
  ProofChain client,
  String userId,
) {
  return client.passports.getWithFields(userId);
}

Future<PassportV2Data> loadPassportView(
  ProofChain client,
  String userId,
  String passportId,
) {
  return client.passports.getPassportV2(userId, passportId);
}
```

`wallets.getInfo` uses camelCase option keys in a `JsonMap` and maps them to wire snake_case. It honors explicit false. `getUserSummary(userId, includeBalances)` uses an optional positional boolean, default true. `getBalance`, `getNFTs`, `getUserNFTs`, and token catalog methods are separate REST reads. Avoid logging entire wallet responses merely for debugging.

`passports.getWithFields` performs two sequential requests: passport followed by field values. It is not a single atomic snapshot. V2 views, milestones, and available views are different model families from the legacy passport object. `getPassportView` delegates to `getPassportV2`. Pass an actual passport definition ID when a specific V2 view is needed; do not assume the same ID is used by the legacy templates API.

Passport history uses `ListOptions(limit: ..., offset: ...)`. Legacy badge and achievement reads (`getUserBadges`, `getUserAchievements`) are distinct from reward and credential records. Do not count one as another without a business rule from your server.

11.3 Backend/admin: provisioning and sensitive wallet operations

Wallet creation/dual creation, NFT registration, token administration, transfers/swaps, key export, and passport points/template/badge/achievement mutations must be restricted to server-authorized workflows. Network selection must match your deployment. Defaults such as `base-mainnet` imply real network resources; never treat the default as a test network.

`users.ensureWallet(externalId, network)` is not a read-only lookup. It may create a missing wallet, and after **any** lookup or single-wallet creation error it falls back to creating a dual pair. Do not call it in a polling loop or as a harmless wallet-existence check. Prefer explicit reads when rendering screens. `wallets.exportKey` posts an acknowledgment flag and returns `JsonMap`; this guide deliberately does not demonstrate exporting or displaying secrets.

Passport `setFieldValue(userId, fieldKey, value)` forwards the value directly, not an invented `{'value': ...}` wrapper. **Current source limitation:** the shared transport omits a body when that top-level value is `false`, `0`, an empty string, or `null`. Do not rely on this helper to clear or disable a field with those scalar values; do not work around it by inventing a wrapper the endpoint does not accept. Only allow application-editable fields, and use recomputation where your deployment defines computed fields. Linking an address does not prove ownership through a signature.

11.4 Backend/admin: schemas, attestations, partner keys, and AirKit

The following small tooling library makes concrete read/session calls without exposing generated partner secrets or invoking revocation/deletion. Its entire surface requires the corresponding authorized server scope. Session creation is an explicit action and must not be called just because a module is imported.

```
import 'package:proofchain_sdk/proofchain.dart';

Future<SchemaDetail> inspectSchemaVersion(
  ProofChain admin,
  String name,
  String version,
) {
  return admin.schemas.get(name, version);
}

Future<AttestationRecord> inspectAttestation(
  ProofChain admin,
  String attestationId,
) {
  return admin.attestations.get(attestationId);
}

Future<OTTConfigResponse> inspectPartnerConfiguration(
  ProofChain admin,
  String keyId,
) {
  return admin.partnerKeys.getOTTConfig(keyId);
}

Future<AirKitSessionResponse> beginAuthorizedAirKitSession(ProofChain admin) {
  return admin.airkit.createSession();
}
```

Schema creation/update take YAML content strings; schema names/versions are not JSON-schema maps. `validateMultiple(schemaNames, data)` submits an array of schema names plus one data object. Activation, deprecation, default selection, and cloning alter tenant configuration and need controlled administration.

`attestations` manages structured attestation records and configuration; it is not `documents.attest`. Public wallet credential methods are `getWalletCredentials`, `getWalletCredential`, and `verifyWalletCredentials`; they use wallet addresses and attestation UIDs rather than document hashes.

`partnerKeys.generate` and `redeem` implement one-time-token workflows defined by the server. Do not log token/key responses or treat them as a mobile login protocol. `getOTTConfig`/`updateOTTConfig` operate on key IDs. AirKit exposes `createSession`; this package does not supply a native AirKit UI or invent a subsequent session exchange protocol. Consume the returned model according to your integration's actual server contract.

12. Native notifications and foreground lifecycle

12.1 Subscribe to native SSE and retain the cancellation handle

`notifications.subscribe` opens a native Server-Sent Events connection and returns an `Unsubscribe` function immediately. It is not a future and should not be awaited. The positional third argument is `SubscribeOptions`. This controller needs only the SDK dependency and makes lifecycle ownership explicit.

```
import 'package:proofchain_sdk/proofchain.dart';

class ForegroundNotifications {
  final ProofChain client;
  final String userId;
  final NotificationCallback onEvent;
  final void Function(String state) onState;
  Unsubscribe? _cancel;

  ForegroundNotifications({
    required this.client,
    required this.userId,
    required this.onEvent,
    required this.onState,
  });

  void resume() {
    if (_cancel != null) return;
    _cancel = client.notifications.subscribe(
      userId,
      onEvent,
      SubscribeOptions(
        onConnect: () => onState('connected'),
        onDisconnect: () => onState('disconnected'),
        onError: (error) => onState('notification parse failure'),
        autoReconnect: true,
        reconnectDelay: 3000,
      ),
    );
  }

  void pause() {
    _cancel?.call();
    _cancel = null;
  }

  void dispose() {
    pause();
  }
}
```

Connect the controller's `resume()` to the application's foreground/resumed lifecycle and call `pause()` when the owner leaves foreground (including hidden, inactive, paused, or detached states according to your application policy). In Flutter, a `WidgetsBindingObserver` or `AppLifecycleListener` in the owning widget/service can forward those transitions; remove that observer/listener in its disposal method. Create one controller per intended subscription owner, not inside `build()`. A controller for a shared client cancels only its subscription; it must not close the entire client when another screen still uses it.

On returning to foreground, refresh authoritative domain data as well as resubscribing. Mobile operating systems can suspend or terminate an application; SSE is not guaranteed background delivery. The client does not persist a notification cursor, send `Last-Event-ID`, or guarantee replay of missed events. Deduplicate/update UI using your domain records rather than assuming exactly-once notification delivery.

12.2 Callback semantics and reconnect behavior

`NotificationEvent` exposes `id`, `event`, `data`, `tenant_id`, `user_id`, and `timestamp`. Treat `data` as event-specific metadata; reload records when a workflow needs authoritative status. Do not mutate privileged server state just because a notification named an event.

The options have deliberately narrow semantics:

Option	Actual behavior
`onConnect`	Called when a parsed JSON payload has `event: connected`, not merely when TCP/HTTP connects
`onDisconnect`	Called after failed connection, stream error, or stream end
`onError`	Reports notification JSON/model parsing or callback-processing failures; not a general transport-error callback
`autoReconnect`	Defaults to true; false closes after a disconnect
`reconnectDelay`	Fixed delay in milliseconds, default 3,000; not exponential backoff

Only unnamed SSE messages or SSE `event: message` are dispatched. Other named SSE event types are ignored, and comment lines are ignored. The JSON `event` field inside an accepted message is distinct from the SSE event-line name. Malformed data is reported through `onError`; the SDK does not forward the underlying connection exception there. Keep callbacks short, synchronous, and nonthrowing; schedule longer application work separately with its own error handling.

Calling the unsubscribe function cancels connecting, connected, and reconnecting work, aborts pending requests, closes the dedicated native transport, and stops its timer. It is safe to call repeatedly. Explicit cancellation does not need a server-side unsubscribe request. `notifications.close()` cancels every active subscription owned by that resource and prevents future subscriptions; it does not close the shared HTTP client. `ProofChain.close()` closes notifications and the shared HTTP transport and is the final owner-level cleanup.

The SSE URL carries query authentication: `api_key` when configured, or `token` and optional `tenant_id` for user authentication. Use HTTPS and avoid logging full URLs in application logs, proxies, analytics, or crash reports. For mobile, use the least-privileged user-token client, never an embedded tenant API key. SSE reconnection reads the client's configured credentials; the main client has no token setter. When credentials change, cancel the old subscription and replace/reconfigure the owning main client through your identity lifecycle.

12.3 Browser Web Push is not native push

The SDK retains server-side Web Push compatibility APIs: `getVapidKey`, `registerPush`, `unregisterPush`, and `generateServiceWorker`. A VAPID key and a `PushSubscriptionJSON` are browser Web Push concepts, not FCM/APNs device tokens. Registering arbitrary native strings as a browser subscription does not enable mobile delivery.

On native Android/iOS, `requestPermissionAndSubscribe` returns null. It does not request OS notification permission, create a service worker, contact `PushManager`, or substitute Firebase Cloud Messaging or Apple Push Notification service. `generateServiceWorker` returns JavaScript source as a string; native Dart does not execute it. `getVapidKey` returns null on any caught failure, so null is not proof that Web Push is intentionally disabled on the server.

Implement native background push separately with your application's actual FCM/APNs infrastructure and backend contract if required. This SDK has no native device-token registration route or permission bridge to demonstrate. Do not promise a background notification merely because foreground SSE works.

These recipes document the implemented source surface. Prior SDK evidence was clean Dart 3.13.5 analysis, 17 behavior tests, and 583 runtime parity scenarios; those are prior implementation checks, not checks run while writing this chapter. Android/iOS builds and real-device behavior remain unverified.

13. Production integration and security

13.1 Ownership and the application boundary

Use a session-scoped `ProofChain` instance, not a new instance in every widget `build()`. It owns the shared native HTTP connection pool and notification subscriptions. Resource objects such as `client.users` are views over that client's transport and credentials, not independently authenticated sessions. Pass the configured client into repositories/services that need it. A method that accepts a caller-owned client should not close it; the session owner closes it on sign-out, replacement, or final disposal.

A foreground notification owner should cancel its `Unsubscribe` callback when the screen/session is no longer active. The SDK does not observe Flutter's lifecycle for you. If your policy pauses streaming when the app leaves the foreground, connect that cancellation to your app's `WidgetsBindingObserver` or existing lifecycle controller, then create a new subscription on resume. Do not assume a native SSE socket will survive background suspension. Renew authentication before reconnecting if the session has expired.

The two ingestion clients do not share the main client, do not have `close()` methods, and own an HTTP transport per operation. `EndUserIngestionClient.setToken()` affects subsequent requests; it does not retroactively change an in-flight request. There is no corresponding token setter on the main `ProofChain` client.

13.2 Keep credentials on the correct side of the boundary

Concern	Application rule
Tenant/admin API keys	Keep in trusted backend/native administrative processes only. Never ship them in app assets, source, build defines or obfuscated binaries.
End-user JWT	Obtain from your existing authenticated backend flow. Use the tenant and user context bound to that session; server authorization remains mandatory.
Token storage	Choose and configure secure storage at the app layer. This SDK does not persist or encrypt credentials for you.
Endpoint selection	Configure approved HTTPS origins. An untrusted `baseUrl`, ingestion `apiUrl`, or `ingestUrl` can receive the credentials you supply.
Logs and diagnostics	Redact `Authorization`, `X-API-Key`, sensitive request/response bodies and notification stream query parameters. Avoid sending personal data to crash reporters.
Logout/session switch	Cancel app-owned work, close the old client, clear app-owned credentials, and recreate only the resources allowed for the new session.
Administration	User deletion, key management, grants, minting, issuance, schema/quest/reward administration and tenant operations require appropriate server-side authorization. Merely having a client method grants nothing.

SSE query authentication is a compatibility constraint. `notifications.subscribe()` builds a stream URL containing `user_id` plus `token`/`tenant_id` for JWT authentication, or `api_key` for API-key authentication. That matches the current JavaScript protocol. The application should not manually assemble or log this URL; configure reverse proxies, telemetry and access-log pipelines to redact these parameters. Never use privileged-key streaming inside a distributed mobile app.

Credentials are sent to the configured origin. The main `ProofChain` shared HTTP transport follows same-origin redirects and refuses authenticated cross-origin redirects. Standalone ingestion and native SSE use separate transports and do not implement that same explicit redirect guard; configure their approved endpoints and proxies not to redirect credentials across origins. Fix routing rather than weakening certificate verification. This SDK provides no certificate-pinning configuration hook.

13.3 Write semantics, duplicates and concurrent edits

The default shared transport permits network-error retries. A connection failure can occur after a server has accepted a mutation but before the client sees the response. Retrying that mutation may duplicate a user, event, issuance, payment-related operation or mint. `maxRetries: 0` disables this shared transport replay; it does not make a write atomic or prove that a failed request had no effect. Do not add blind outer retries around writes. Reconcile with authoritative server state or use an explicit backend-provided idempotency contract when that endpoint actually supports one; the SDK does not invent an idempotency key protocol.

The following helpers are not transactions:

- `events.createBatch()` calls individual creates sequentially. If an intermediate request fails, earlier events may already exist. It is not the standalone ingestion batch endpoint.
- Quest step helpers read the quest and replace its steps array. Concurrent administrators can overwrite each other's edits. Serialize editing or use a backend concurrency policy; no optimistic version check is added by the SDK.
- `quests.reorderSteps()` submits the IDs you pass. Supply the complete intended set in the desired order; omitted steps are not automatically preserved by that replacement. Unknown IDs throw before the update, but the helper is not a general duplicate/completeness validator.
- `users.removeAttributes()` reads attributes and patches a modified object. Coordinate concurrent updates rather than treating it as a field-level atomic removal endpoint.
- `users.ensureWallet()` may create wallets. Its inherited JavaScript behavior falls back to dual-wallet creation after lookup/single-wallet errors, not only after a not-found result. Do not treat it as a side-effect-free probe or proof of exactly-once provisioning.

Concurrent identical GET requests on one main client share an in-flight request. Once it settles, the entry is removed: this is neither persistent response caching nor a freshness/TTL policy. Mutations are not coalesced, and separate `ProofChain` instances do not share that in-flight map.

13.4 Timeouts, memory and binary data

Path	Timeout/retry behavior
Main JSON and multipart methods	`timeout` is milliseconds, default 30000; connection/request/header deadline, not a response-body deadline. Network-error retry loop only; typed timeout and HTTP error responses are not replayed by that loop.
Vault binary download	Uses the connection/header deadline, buffers response bytes, and does not use the JSON transport's retry loop. Non-2xx responses become `ProofChainError`; transport/timeout failures are wrapped as `NetworkError`.
`users.export()`	Uses configured native URL/authentication and returns bytes. Like the source implementation, HTTP error statuses are not converted to typed exceptions; error-response bytes can therefore be returned.
Ingestion single request	Whole-operation deadline, no retries. Ingest timeout is surfaced as `NetworkError`.
Ingestion batch	Twice the configured ingestion timeout, no retries. Batch-size validation happens before network I/O.
Direct `getEventStatus()`	Whole-operation deadline; timeout is a `dart:async` `TimeoutException`, unlike ingest/ingestBatch.
Notifications	Separate SSE lifecycle and reconnect options. Do not assume the main client's JSON timeout describes the entire lifetime of a stream.

A `Future.timeout()` in application code can stop waiting without canceling the underlying SDK request. It is not a rollback mechanism and does not provide a public per-request abort handle. Avoid presenting a timed-out mutation as certainly uncommitted.

`UploadFile` takes an in-memory byte list. Vault downloads and user exports return a fully buffered `Uint8List`. There is no public chunked upload, download-to-file stream, progress callback or resume API. Size and validate files before reading them into memory; choose app-specific limits based on device memory and backend policy. File picking, media permissions, temporary-directory selection and opening/sharing a downloaded file belong to the app and its chosen plugins. SDK networking itself does not require adding camera or broad storage permissions.

14. JavaScript migration and troubleshooting

14.1 Mechanical migration rules

JavaScript concept	Dart implementation
Main package import	`import 'package:proofchain_sdk/proofchain.dart';`
Named interface argument	A model with named constructor parameters, such as `UserEventHistoryOptions(days: 3, limit: 50)`.
Anonymous object / `Partial<T>` input	`JsonMap`, retaining the actual expected wire keys.
Optional positional argument	Remains positional. Do not turn `byUser(userId, options)` into `byUser(userId, options: options)`.
Promise	`Future<T>`; use `await` and typed exception handling.
File / Blob / Buffer input	`UploadFile(bytes, filename: ..., contentType: ...)`, often inside a typed request. Endpoint request-level filename fields can override the file's name.
Binary result	`Uint8List`; storage/display is the app's responsibility.
String-literal union	Dart `String` alias. The server still validates allowed values.
JSON key casing	Preserved, including snake_case. Do not mechanically camel-case every field.
Reserved Dart member	A `Value` suffix where required, e.g. `SchemaField.defaultValue` serializes under `default`.
Browser notifications	Native SSE is implemented. Web Push registration/service-worker source methods remain; the browser permission helper returns null on native.
Embedded wallet provider	Not ported by this package. Main SDK wallet REST methods remain available.

`fromJson()` retains the original wire map, including unknown fields and explicit nulls. A normal named constructor omits optional null fields in `toJson()`. Use a `JsonMap` update or a request's `fromJson()` when an explicit JSON null is the intended payload. Do not rely on mutating a response's raw map to update its already decoded typed getters; treat response models and their raw maps as read-only snapshots.

Not all similarly named types have the same purpose. `ValidationError` is an HTTP exception; `SchemaValidationError` is a schema-validation model. `AttestationRecord` is distinct from document `AttestationResult`. User wallet creation uses `CreateUserWalletRequest`; wallet resource creation uses its own request type. Reward verification is `RewardVerifyResult`. These names resolve collisions in the exported JavaScript surface.

14.2 Troubleshooting by symptom

Symptom	Check / corrective action
Package import cannot be resolved	Verify the registry constraint or path relative to the app's pubspec, run `flutter pub get` in the app, and use `package:proofchain_sdk/proofchain_sdk.dart`. GitHub source changes are not available through pub until a new version is published.
Build targets a browser	This package imports `dart:io`; choose Android/iOS/native Dart. A browser-compatible transport is not included.
Works in debug but not Android release	Verify `INTERNET` is in the main manifest, not only a debug variant. Inspect the release build's actual endpoint and certificate trust.
401	Check JWT expiration or the trusted process's API key. Replace the main client after token refresh. Do not print credentials while debugging.
403	Check server-side authorization, tenant identity and user scope. Do not insert an admin key into the mobile app. Ingestion maps some non-401/validation/rate/server statuses to the base `ProofChainError`, rather than the main transport's specialized class.
Wrong tenant or route/404	Distinguish `tenant` slug from `tenantId`; explicitly configure the correct base path, including `/api` when the deployment requires it. External and internal user IDs are not interchangeable.
Validation details are not `{field, message}`	Inspect/redact `ValidationError.rawErrors`; do not unconditionally read the typed `errors` getter on an arbitrary server detail shape.
429 is not retried	Expected current source behavior. `retryAfter` is seconds when numeric; null or non-finite values need an application fallback. HTTP-date Retry-After values are not parsed as dates.
Request exceeds the configured milliseconds	Main timeout stops at response headers; retries add elapsed time, and response-body reading has no main whole-operation deadline.
Export bytes do not decode as expected CSV	The inherited export method can return an HTTP error body as bytes. Do not equate a nonempty buffer with successful authorized export or show raw bytes as user-facing error text.
Required response getter throws	Check raw `toJson()` and API version/response shape. The typed source interfaces can require keys omitted by the server; a typed getter is not a runtime completeness guarantee.
Notification stream is silent/disconnects	Verify authorized user/tenant, proxy SSE support, server connectivity and app foreground state. Inspect sanitized callbacks. Do not assume background push or automatic token refresh.
Duplicate writes after a connectivity failure	Reconcile server state before replay. Disable automatic network retries for sensitive sessions/operations where the backend lacks a safe idempotency contract.

Symptom	Check / corrective action
Local development endpoint is unreachable from a device	The device's loopback host is not your development workstation. Use your platform's documented simulator/emulator networking and an approved reachable development endpoint; avoid globally weakening TLS.

14.3 Platform behavior that must not be mistaken for parity

The port mirrors the current local JavaScript API, not every browser capability or every possible backend version. Native `users.export()` uses the SDK's configured base URL and credentials instead of browser-relative fetch and returns `Uint8List`. The main client's shared HTTP transport deliberately refuses authenticated cross-origin redirects; this is not a blanket guarantee for standalone ingestion or SSE. Dart URI normalization may render RFC3986-equivalent unreserved characters differently without changing the URL's meaning.

`requestPermissionAndSubscribe()` does not request Android/iOS notification permission and returns null. Generated service-worker source is for a separate browser deployment, not a native isolate. FCM/APNs integration, reliable background delivery, embedded wallet signing/provider UX, login/token refresh, offline persistence/queues and Flutter Web are not supplied by this package. Do not label an app as supporting these simply because an adjacent REST method exists.

15. Development, verification and release checklist

15.1 SDK checks

From the Flutter SDK package root, with Dart on PATH:

```
dart pub get
dart analyze
dart test
dart format --output=none --set-exit-if-changed lib example test
```

The maintainer-only model generator requires the upstream JavaScript SDK package at `../javascript`, including its `src/` tree and installed TypeScript development dependency. That upstream checkout is not bundled in the standalone public Flutter repository and is not needed to consume the SDK or generate Dart API documentation. Run the first command in the upstream JavaScript package and the remaining commands in the Flutter SDK package:

```
npm ci
```

```
node tool/generate_models.mjs --check
```

After reviewing changes to JavaScript interfaces, regenerate intentionally:

```
node tool/generate_models.mjs
```

`DART=/absolute/path/to/dart` selects the formatter used by the model generator. Generated models must not be edited by hand. Generation covers model declarations; it does not automatically implement newly added JavaScript resource methods. Review method signatures, URL/body mapping, return decoding and observable error behavior whenever the reference SDK changes.

15.2 Rebuild this guide

The documentation builder has its own private Dart package under `tool/docs`; its analyzer/Markdown dependencies are not runtime dependencies of `proofchain_sdk`. It requires Dart 3.13 or newer and a Chromium-family executable for PDF generation. The example checker statically analyzes standalone native Dart code blocks from these chapters; Flutter-importing blocks are explicitly reported as not checked by that native-only step.

From the package's `tool/docs` directory:

```
dart pub get
dart analyze
dart test
dart run build_guide.dart --check-examples --pdf
```

Set `CHROME_BIN` to your Chromium/Chrome executable path if it is not named `chromium`. The builder refreshes the source-derived API reference, JSON reference manifest, self-contained HTML and `docs/proofchain-flutter-sdk-developer-guide.pdf`. Run `dart run build_guide.dart` without `--pdf` to regenerate the editable/viewable HTML and reference without launching a renderer. Author chapters 01–03 and `docs/print.css`; chapter 04 is generated. Treat the PDF as an output artifact, not the editable source.

15.3 Before releasing an app

- Use a reviewed SDK checkout and verify the app's actual release dependency resolution.
- Analyze and test the app; run its Android/iOS release builds on the appropriate host/toolchains. iOS signing and distribution require the corresponding Apple environment.
- Exercise real tenant/user authorization in an approved test environment: expired token, wrong tenant, forbidden operation, missing resource, validation and rate-limit paths.
- Verify the main Android manifest and the configured HTTPS deployment on real devices; do not substitute desktop HTTP fixture tests for platform checks.
- Exercise foreground/background/resume, account switch and logout with SSE active, including reconnection after the network changes.
- Check that stale asynchronous results cannot update disposed widgets or a newly authenticated session.
- Verify upload/download sizes, filenames and error handling under realistic device memory and network conditions.
- Ensure no API key is distributed and no JWT or SSE query credential appears in app/proxy/crash logs.
- Reconcile ambiguous write failures before enabling retry/replay behavior; verify any backend idempotency promises separately.
- Pin a reviewed published SDK release, or explicitly select a reviewed path/Git checkout. A GitHub push does not replace an immutable pub.dev release.

The prior native implementation verification establishes local behavior and source compatibility, not production deployment readiness. Documentation checks establish that the examples agree with current Dart declarations and that the PDF renders; they do not authenticate a live tenant, mint a real token, provision a wallet or validate a device build.

15.4 Initial guide verification (0.1.0)

These results were recorded for the initial guide edition. They are historical evidence, not a substitute for running the current checkout's release checks.

Check	Observed result
Documentation tool analysis	Dart 3.13.5; no issues
Renderer-security regression tests	3 passed; active HTML/assets are removed and declaration/example text remains literal
Native Dart example analysis	All 28 standalone examples passed against this SDK checkout
Executed sample workflows	10 real loopback HTTP/SSE requests covering initialization, profile updates, two-page pagination, multipart/binary round-trip, end-user ingestion, notification lifecycle, environment configuration and safe CLI failure boundaries
Flutter screen / Android/iOS	Not built or device-tested; Flutter toolchain unavailable in this documentation session
Live service / production credentials	Not used; smoke scenarios used fixture credentials and local servers only

The PDF is an implementation guide, not a claim of device certification, production-tenant access, background push integration or package publication.

16. API reference

This reference is generated from the Dart syntax tree, not handwritten method lists. Names, parameter positions, defaults and return types are the current source declarations. Signatures are reference notation, not standalone Dart programs. All `client.*` resources share the configuration and authentication of their owning `ProofChain` instance. Method availability does not grant authorization: the server must allow the requested operation for the current identity.

`JsonMap` means `Map<String, Object?>`; `UploadFile` carries bytes, a filename and content type; downloads use `Uint8List`. Optional positional parameters appear inside `[...]`; named parameters inside `{...}`. `Future<T>` must be awaited. A `JsonMap` return value has no stronger SDK field guarantee than the current server response.

16.1 Construction, lifecycle and top-level operations

```
ProofChain(ProofChainOptions options)
```

```
factory ProofChain.forEndUser({required String userToken, required String tenantId, String? baseUrl, num? timeout, num? maxRetries})
```

```
factory ProofChain.fromEnv()
```

`ProofChain.fromEnv()` reads `PROOFCHAIN_API_KEY` and optional `PROOFCHAIN_BASE_URL` only. Missing/empty API key throws `StateError`. `tenantApiUrl(String slug)` returns `https://<slug>.proofchain.co.za/api`; it constructs a URL and does not discover, create or authenticate a tenant.

`client.verify`

```
Future<VerificationResult> verify(String ipfsHash)
```

`client.tenantInfo`

```
Future<TenantInfo> tenantInfo()
```

`client.usage`

```
Future<UsageStats> usage([String period = 'month'])
```

`client.close` — native lifecycle

```
void close()
```

client.documents

Multipart document attestation and hash lookup.

Source: `lib/src/client.dart` · Class: `DocumentsResource`.

`client.documents.attest`

```
Future<AttestationResult> attest(AttestRequest request)
```

`client.documents.get`

```
Future<Event> get(String ipfsHash)
```

client.events

Create, list, search and retrieve tenant events.

Source: `lib/src/client.dart` · Class: `EventsResource`.

`client.events.create`

```
Future<Event> create(CreateEventRequest request)
```

`client.events.get`

```
Future<Event> get(String eventId)
```

`client.events.list`

```
Future<List<Event>> list([ListEventsRequest request = const ListEventsRequest()])
```

`client.events.search`

```
Future<SearchResult> search(SearchRequest request)
```

`client.events.byHash`

```
Future<Event> byHash(String ipfsHash)
```

`client.events.createBatch`

```
Future<List<Event>> createBatch(List<CreateEventRequest> events)
```

client.channels

Channel lifecycle, streaming and settlement.

Source: `lib/src/client.dart` · Class: `ChannelsResource`.

`client.channels.create`

```
Future<Channel> create(CreateChannelRequest request)
```

`client.channels.get`

```
Future<Channel> get(String channelId)
```

`client.channels.status`

```
Future<ChannelStatus> status(String channelId)
```

`client.channels.list`

```
Future<List<Channel>> list([num limit = 50, num offset = 0])
```

`client.channels.stream`

```
Future<StreamAck> stream(String channelId, StreamEventRequest request)
```

`client.channels.streamBatch`

```
Future<JsonMap> streamBatch(String channelId, List<StreamEventRequest> events)
```

`client.channels.settle`

```
Future<Settlement> settle(String channelId)
```

`client.channels.close`

```
Future<Channel> close(String channelId)
```

client.certificates

Certificate issuance, lookup, revocation and verification.

Source: `lib/src/client.dart` · Class: `CertificatesResource`.

`client.certificates.issue`

```
Future<Certificate> issue(IssueCertificateRequest request)
```

`client.certificates.get`

```
Future<Certificate> get(String certificateId)
```

`client.certificates.list`

```
Future<List<Certificate>> list([ListCertificatesRequest request = const
ListCertificatesRequest()])
```

`client.certificates.revoke`

```
Future<Certificate> revoke(String certificateId, [String? reason])
```

`client.certificates.verify`

```
Future<JsonMap> verify(String certificateId)
```

client.webhooks

Manage and test server-to-server webhooks.

Source: `lib/src/client.dart` · Class: `WebhooksResource`.

`client.webhooks.create`

```
Future<Webhook> create(CreateWebhookRequest request)
```

`client.webhooks.get`

```
Future<Webhook> get(String webhookId)
```

`client.webhooks.list`

```
Future<List<Webhook>> list()
```

`client.webhooks.update`

```
Future<Webhook> update(String webhookId, UpdateWebhookRequest request)
```

`client.webhooks.delete`

```
Future<void> delete(String webhookId)
```

`client.webhooks.test`

```
Future<JsonMap> test(String webhookId)
```

client.vault

Store, list, download, retrieve and remove vault documents.

Source: `lib/src/resources.dart` · Class: `VaultResource`.

`client.vault.list`

```
Future<VaultListResponse> list([String? folderId])
```

`client.vault.upload`

```
Future<VaultFile> upload(VaultUploadRequest request)
```

`client.vault.get`

```
Future<VaultFile> get(String fileId)
```

`client.vault.download`

```
Future<Uint8List> download(String fileId)
```

`client.vault.delete`

```
Future<void> delete(String fileId)
```

`client.vault.move`

```
Future<VaultFile> move(String fileId, String folderId)
```

`client.vault.createFolder`

```
Future<VaultFolder> createFolder(String name, [String? parentId])
```

`client.vault.deleteFolder`

```
Future<void> deleteFolder(String folderId)
```

`client.vault.stats`

```
Future<VaultStats> stats()
```

`client.vault.share`

```
Future<JsonMap> share(String fileId, [num? expiresInHours])
```

client.search

Tenant search, user event history and event counts.

Source: `lib/src/resources.dart` · Class: `SearchResource`.

`client.search.query`

```
Future<SearchResponse> query([SearchQueryRequest? request])
```

`client.search.quick`

```
Future<SearchResponse> quick(String query, [num limit = 20])
```

`client.search.byUser`

```
Future<UserEventHistoryResponse> byUser(String userId, [Object? optionsOrLimit, num? offset])
```

`client.search.byCertificate`

```
Future<Event> byCertificate(String certificateId)
```

`client.search.facets`

```
Future<FacetsResponse> facets([String? fromDate, String? toDate])
```

`client.search.stats`

```
Future<SearchStats> stats()
```

client.verifyResource

Verification lookups and proof operations.

Source: `lib/src/resources.dart` · Class: `VerifyResource`.

`client.verifyResource.certificate`

```
Future<CertificateVerifyResult> certificate(String certificateId)
```

`client.verifyResource.event`

```
Future<JsonMap> event(String ipfsHash)
```

`client.verifyResource.proof`

```
Future<ProofVerifyResult> proof(String leaf, List<String> proof, String root)
```

`client.verifyResource.batch`

```
Future<BatchVerifyResult> batch(String batchId)
```

`client.verifyResource.eventBatchProof`

```
Future<EventBatchProof> eventBatchProof(String eventId)
```

`client.verifyResource.batchVerify`

```
Future<JsonMap> batchVerify(List<JsonMap> items)
```

client.tenant

Tenant context, API-key administration, usage and blockchain operations.

Source: `lib/src/resources.dart` · Class: `TenantResource`.

`client.tenant.listApiKeys`

```
Future<List<ApiKey>> listApiKeys()
```

`client.tenant.createApiKey`

```
Future<ApiKey> createApiKey(CreateApiKeyRequest request)
```

`client.tenant.deleteApiKey`

```
Future<void> deleteApiKey(String keyId)
```

`client.tenant.usageDetailed`

```
Future<JsonMap> usageDetailed([String? fromDate, String? toDate])
```

`client.tenant.context`

```
Future<JsonMap> context()
```

`client.tenant.blockchainStats`

```
Future<BlockchainStats> blockchainStats()
```

`client.tenant.blockchainVerify`

```
Future<BlockchainProof> blockchainVerify(String certificateId)
```

`client.tenant.blockchainCertificates`

```
Future<JsonMap> blockchainCertificates([num? limit, num? offset])
```

`client.tenant.forceBatch`

```
Future<JsonMap> forceBatch()
```

`client.tenant.settleAll`

```
Future<JsonMap> settleAll()
```

`client.tenant.settleEvent`

```
Future<JsonMap> settleEvent(String eventId)
```

client.passports

User passports, categories, linked events and grants.

Source: `lib/src/passports.dart` · Class: `PassportClient`.

`client.passports.list`

```
Future<List<Passport>> list([ListOptions options = const ListOptions()])
```

`client.passports.get`

```
Future<Passport> get(String userId)
```

`client.passports.getWithFields`

```
Future<PassportWithFields> getWithFields(String userId)
```

`client.passports.create`

```
Future<Passport> create(CreatePassportRequest data)
```

`client.passports.update`

```
Future<Passport> update(String userId, UpdatePassportRequest data)
```

`client.passports.delete`

```
Future<void> delete(String userId)
```

`client.passports.addPoints`

```
Future<Passport> addPoints(String userId, num points, [String? reason])
```

`client.passports.levelUp`

```
Future<Passport> levelUp(String userId)
```

`client.passports.linkWallet`

```
Future<Passport> linkWallet(String userId, String walletAddress)
```

`client.passports.getFieldValues`

```
Future<List<FieldValue>> getFieldValues(String userId)
```

`client.passports.setFieldValue`

```
Future<void> setFieldValue(String userId, String fieldKey, Object? value)
```

client.passports.recomputeFields

```
Future<JsonMap> recomputeFields(String userId)
```

client.passports.assignTemplate

```
Future<void> assignTemplate(String userId, String templateId)
```

client.passports.listTemplates

```
Future<List<PassportTemplate>> listTemplates()
```

client.passports.getTemplate

```
Future<PassportTemplate> getTemplate(String templateId)
```

client.passports.createTemplate

```
Future<PassportTemplate> createTemplate(CreateTemplateRequest data)
```

client.passports.addTemplateField

```
Future<TemplateField> addTemplateField(String templateId, CreateTemplateFieldRequest field)
```

client.passports.deleteTemplate

```
Future<void> deleteTemplate(String templateId)
```

client.passports.listBadges

```
Future<List<Badge>> listBadges()
```

client.passports.createBadge

```
Future<Badge> createBadge(CreateBadgeRequest data)
```

client.passports.awardBadge

```
Future<UserBadge> awardBadge(String userId, String badgeId, [JsonMap? metadata])
```

client.passports.getUserBadges

```
Future<List<UserBadge>> getUserBadges(String userId)
```

client.passports.listAchievements

```
Future<List<Achievement>> listAchievements()
```

client.passports.createAchievement

```
Future<Achievement> createAchievement(CreateAchievementRequest data)
```

client.passports.getUserAchievements

```
Future<List<UserAchievement>> getUserAchievements(String userId)
```

client.passports.updateAchievementProgress

```
Future<UserAchievement> updateAchievementProgress(String userId, String achievementId, num
progress)
```

client.passports.getHistory

```
Future<List<PassportHistory>> getHistory(String userId, [ListOptions options = const
ListOptions()])
```

client.passports.getPassportV2

```
Future<PassportV2Data> getPassportV2(String userId, [String? passportId])
```

client.passports.getPassportView

```
Future<PassportV2Data> getPassportView(String userId, [String? passportId])
```

client.passports.getMilestones

```
Future<List<Milestone>> getMilestones(String userId, [String? passportId])
```

`client.passports.listAvailableViews`

```
Future<List<AvailableView>> listAvailableViews()
```

`client.passports.listPassportDefinitions`

```
Future<List<PassportDefinition>> listPassportDefinitions()
```

`client.passports.createPassportDefinition`

```
Future<PassportDefinition> createPassportDefinition(CreatePassportDefinitionRequest definition)
```

client.wallets

Wallet REST operations; not a local signing/provider implementation.

Source: `lib/src/wallets.dart` · Class: `WalletClient`.

`client.wallets.create`

```
Future<Wallet> create(CreateWalletRequest data)
```

`client.wallets.get`

```
Future<Wallet> get(String walletId)
```

`client.wallets.listByUser`

```
Future<List<Wallet>> listByUser(String userId)
```

`client.wallets.stats`

```
Future<WalletStats> stats()
```

`client.wallets.listUsersWithWallets`

```
Future<UsersWithWalletsResponse> listUsersWithWallets([num limit = 50, num offset = 0])
```

client.wallets.createDual

```
Future<DualWallets> createDual(CreateDualWalletsRequest data)
```

client.wallets.createDualBulk

```
Future<List<DualWallets>> createDualBulk(List<String> userIds, [String network = 'base-
mainnet'])
```

client.wallets.getBalance

```
Future<WalletBalanceResponse> getBalance(String walletId)
```

client.wallets.getInfo

```
Future<ComprehensiveWalletInfo> getInfo(String walletId, [JsonMap options = const {}])
```

client.wallets.getUserSummary

```
Future<UserWalletSummary> getUserSummary(String userId, [bool includeBalances = true])
```

client.wallets.exportKey

```
Future<JsonMap> exportKey(String walletId)
```

client.wallets.transfer

```
Future<TransferResult> transfer(TransferRequest request)
```

client.wallets.getSwapQuote

```
Future<SwapQuote> getSwapQuote(SwapQuoteRequest request)
```

client.wallets.executeSwap

```
Future<SwapResult> executeSwap(ExecuteSwapRequest request)
```

client.wallets.getNFTs

```
Future<List<NFT>> getNFTs(String walletId)
```

client.wallets.getUserNFTs

```
Future<List<NFT>> getUserNFTs(String userId)
```

client.wallets.addNFT

```
Future<NFT> addNFT(String walletId, AddNFTRequest nft)
```

client.wallets.getTransactions

```
Future<TransactionHistory> getTransactions(String walletId, [JsonMap options = const {}])
```

client.wallets.createToken

```
Future<Token> createToken(CreateTokenRequest request)
```

client.wallets.listTokens

```
Future<List<Token>> listTokens([JsonMap options = const {}])
```

client.wallets.listGlobalTokens

```
Future<List<Token>> listGlobalTokens([String? network])
```

client.wallets.getToken

```
Future<Token> getToken(String tokenId)
```

client.wallets.getTokenByContract

```
Future<Token> getTokenByContract(String contractAddress, String network)
```

client.wallets.updateToken

```
Future<Token> updateToken(String tokenId, UpdateTokenRequest request)
```

`client.wallets.deleteToken`

```
Future<JsonMap> deleteToken(String tokenId)
```

client.users

End-user lifecycle, profiles, linked wallets, attributes, GDPR and exports.

Source: `lib/src/users.dart` · Class: `EndUsersClient`.

`client.users.list`

```
Future<EndUserListResponse> list([ListEndUsersOptions options = const ListEndUsersOptions()])
```

`client.users.get`

```
Future<EndUser> get(String externalId)
```

`client.users.getByInternalId`

```
Future<EndUser> getByInternalId(String userId)
```

`client.users.create`

```
Future<EndUser> create(CreateEndUserRequest data)
```

`client.users.update`

```
Future<EndUser> update(String externalId, UpdateEndUserRequest data)
```

`client.users.delete`

```
Future<void> delete(String userId)
```

`client.users.linkWallet`

```
Future<EndUser> linkWallet(String externalId, LinkWalletRequest request)
```

client.users.getActivity

```
Future<UserActivityResponse> getActivity(String externalId, [JsonMap options = const {}])
```

client.users.addPoints

```
Future<EndUser> addPoints(String externalId, num points, [String? reason])
```

client.users.merge

```
Future<EndUser> merge(MergeUsersRequest request)
```

client.users.updateByInternalId

```
Future<EndUser> updateByInternalId(String userId, UpdateEndUserRequest data)
```

client.users.getRewards

```
Future<UserRewardsResponse> getRewards(String externalId, [JsonMap options = const {}])
```

client.users.getRewardsByInternalId

```
Future<UserRewardsResponse> getRewardsByInternalId(String userId, [JsonMap options = const {}])
```

client.users.createWallet

```
Future<WalletCreationResult> createWallet(String externalId, [CreateUserWalletRequest request =
const CreateUserWalletRequest()])
```

client.users.registerWallet

```
Future<WalletCreationResult> registerWallet(String externalId, RegisterWalletRequest request)
```

client.users.ensureWallet

```
Future<EnsureWalletResult> ensureWallet(String externalId, [String network = 'base-mainnet'])
```

`client.users.updateAttributes`

```
Future<EndUser> updateAttributes(String externalId, JsonMap attributes)
```

`client.users.removeAttributes`

```
Future<EndUser> removeAttributes(String externalId, List<String> keys)
```

`client.users.setProfile`

```
Future<EndUser> setProfile(String externalId, SetProfileRequest profile)
```

`client.users.gdprPreview`

```
Future<GDPRPreviewResponse> gdprPreview(String userId)
```

`client.users.gdprDelete`

```
Future<GDPRDeletionResponse> gdprDelete(String userId, GDPRDeletionRequest request)
```

`client.users.export`

```
Future<Uint8List> export([JsonMap options = const {}])
```

client.rewards

Reward definitions, assignment, redemption and verification.

Source: `lib/src/rewards.dart` · Class: `RewardsClient`.

`client.rewards.listDefinitions`

```
Future<List<RewardDefinition>> listDefinitions([ListRewardsOptions? options])
```

`client.rewards.getDefinition`

```
Future<RewardDefinition> getDefinition(String definitionId)
```

`client.rewards.createDefinition`

```
Future<RewardDefinition> createDefinition(CreateRewardDefinitionRequest data)
```

`client.rewards.updateDefinition`

```
Future<RewardDefinition> updateDefinition(String definitionId, JsonMap data)
```

`client.rewards.deleteDefinition`

```
Future<void> deleteDefinition(String definitionId)
```

`client.rewards.activateDefinition`

```
Future<RewardDefinition> activateDefinition(String definitionId)
```

`client.rewards.deactivateDefinition`

```
Future<RewardDefinition> deactivateDefinition(String definitionId)
```

`client.rewards.listEarned`

```
Future<List<EarnedReward>> listEarned([JsonMap options = const {}])
```

`client.rewards.getUserRewards`

```
Future<List<RewardUserReward>> getUserRewards(String userId, [JsonMap options = const {}])
```

`client.rewards.getCatalog`

```
Future<List<RewardDefinition>> getCatalog([String? category])
```

`client.rewards.claimNFT`

```
Future<ClaimNFTResult> claimNFT(String earnedRewardId, String walletAddress)
```

`client.rewards.claimReward`

```
Future<ClaimNFTResult> claimReward(String earnedRewardId, String walletAddress)
```

`client.rewards.award`

```
Future<List<EarnedReward>> award(ManualRewardRequest request)
```

`client.rewards.awardManual`

```
Future<List<EarnedReward>> awardManual(ManualRewardRequest request)
```

`client.rewards.distribute`

```
Future<DistributeResult> distribute(String earnedRewardId)
```

`client.rewards.distributePending`

```
Future<DistributeResult> distributePending(String earnedRewardId)
```

`client.rewards.distributeAll`

```
Future<JsonMap> distributeAll()
```

`client.rewards.revoke`

```
Future<RevokeResult> revoke(String earnedRewardId, String reason)
```

`client.rewards.uploadAsset`

```
Future<RewardAsset> uploadAsset(String definitionId, UploadFile file, String assetType)
```

`client.rewards.listAssets`

```
Future<List<RewardAsset>> listAssets(String definitionId)
```

`client.rewards.attest`

```
Future<RewardAttestationResult> attest(String earnedRewardId)
```

`client.rewards.anchor`

```
Future<AnchorResult> anchor(String earnedRewardId)
```

`client.rewards.getAttestation`

```
Future<RewardAttestationResult> getAttestation(String earnedRewardId)
```

`client.rewards.verifyAttestation`

```
Future<RewardVerifyResult> verifyAttestation(String earnedRewardId)
```

client.quests

Quest definitions, steps, user progress and completion.

Source: `lib/src/quests.dart` · Class: `QuestsClient`.

`client.quests.list`

```
Future<List<Quest>> list([ListQuestsOptions options = const ListQuestsOptions()])
```

`client.quests.get`

```
Future<Quest> get(String questId)
```

`client.quests.listAvailable`

```
Future<List<QuestWithProgress>> listAvailable(String userId, [JsonMap options = const {}])
```

`client.quests.create`

```
Future<Quest> create(CreateQuestRequest data)
```

`client.quests.update`

```
Future<Quest> update(String questId, JsonMap data)
```

`client.quests.delete`

```
Future<void> delete(String questId)
```

`client.quests.publish`

```
Future<Quest> publish(String questId)
```

`client.quests.activate`

```
Future<Quest> activate(String questId)
```

`client.quests.pause`

```
Future<Quest> pause(String questId)
```

`client.quests.getWithProgress`

```
Future<QuestWithProgress> getWithProgress(String questId, String userId)
```

`client.quests.listWithProgress`

```
Future<List<QuestWithProgress>> listWithProgress(String userId, [ListQuestsOptions options =
const ListQuestsOptions()])
```

`client.quests.startQuest`

```
Future<UserQuestProgress> startQuest(String questId, String userId)
```

`client.quests.resolveCtaLink`

```
Future<QuestCtaLink> resolveCtaLink(Object quest, [String? userId])
```

`client.quests.listCtaResolvers`

```
Future<List<QuestCtaResolver>> listCtaResolvers()
```

`client.quests.getUserProgress`

```
Future<UserQuestProgress> getUserProgress(String questId, String userId)
```

`client.quests.startStep`

```
Future<StepStartResult> startStep(String questId, String userId, num stepIndex)
```

`client.quests.completeStep`

```
Future<StepCompletionResult> completeStep(String questId, String userId, num stepIndex)
```

`client.quests.getAllUserProgress`

```
Future<List<UserQuestProgress>> getAllUserProgress(String userId)
```

`client.quests.claimReward`

```
Future<ClaimRewardResult> claimReward(String questId, String userId)
```

`client.quests.addStep`

```
Future<QuestStep> addStep(String questId, CreateQuestStepRequest step)
```

`client.quests.updateStep`

```
Future<QuestStep> updateStep(String questId, String stepId, JsonMap data)
```

`client.quests.deleteStep`

```
Future<void> deleteStep(String questId, String stepId)
```

`client.quests.reorderSteps`

```
Future<Quest> reorderSteps(String questId, List<String> stepIds)
```

client.schemas

Schema definitions, validation and inheritance.

Source: `lib/src/schemas.dart` · Class: `SchemasClient`.

`client.schemas.list`

```
Future<SchemaListResponse> list([ListSchemasOptions? options])
```

`client.schemas.get`

```
Future<SchemaDetail> get(String name, [String? version])
```

`client.schemas.create`

```
Future<SchemaDetail> create(String yamlContent)
```

`client.schemas.update`

```
Future<SchemaDetail> update(String name, String yamlContent)
```

`client.schemas.delete`

```
Future<void> delete(String name, [String? version])
```

`client.schemas.activate`

```
Future<Schema> activate(String name, [String? version])
```

`client.schemas.deprecate`

```
Future<Schema> deprecate(String name, [String? version])
```

`client.schemas.setDefault`

```
Future<Schema> setDefault(String name, String version)
```

`client.schemas.validate`

```
Future<ValidationResult> validate(ValidateDataRequest request)
```

`client.schemas.validateMultiple`

```
Future<List<ValidationResult>> validateMultiple(List<String> schemaNames, JsonMap data)
```

`client.schemas.getUsageStats`

```
Future<JsonMap> getUsageStats(String name)
```

`client.schemas.clone`

```
Future<SchemaDetail> clone(String sourceName, String newName, [String? newVersion])
```

client.dataViews

Data-view definitions, previews and execution.

Source: `lib/src/dataviews.dart` · Class: `DataViewsClient`.

`client.dataViews.list`

```
Future<DataViewListResponse> list()
```

`client.dataViews.get`

```
Future<DataViewDetail> get(String viewName)
```

`client.dataViews.create`

```
Future<DataViewDetail> create(CreateDataViewRequest request)
```

`client.dataViews.update`

```
Future<DataViewDetail> update(String viewName, UpdateDataViewRequest request)
```

`client.dataViews.delete`

```
Future<void> delete(String viewName)
```

`client.dataViews.execute`

```
Future<DataViewExecuteResult> execute(String identifier, String viewName)
```

`client.dataViews.preview`

```
Future<DataViewPreviewResult> preview(DataViewPreviewRequest request)
```

`client.dataViews.getFanProfile`

```
Future<FanProfileView> getFanProfile(String walletAddress)
```

`client.dataViews.getActivitySummary`

```
Future<ActivitySummaryView> getActivitySummary(String walletAddress, [num days = 30])
```

`client.dataViews.getEventMetadata`

```
Future<EventMetadata> getEventMetadata()
```

`client.dataViews.getTemplates`

```
Future<List<ViewTemplate>> getTemplates()
```

client.cohorts

Cohort membership and user/cohort leaderboard queries.

Source: `lib/src/cohorts.dart` · Class: `CohortLeaderboardClient`.

`client.cohorts.list`

```
Future<List<CohortDefinition>> list([ListCohortsOptions? options])
```

`client.cohorts.get`

```
Future<CohortDefinition> get(String cohortId)
```

`client.cohorts.getLeaderboard`

```
Future<CohortLeaderboardResponse> getLeaderboard(String cohortId, [CohortLeaderboardOptions?
options])
```

`client.cohorts.getUserBreakdown`

```
Future<UserBreakdownResponse> getUserBreakdown(String userId, [JsonMap options = const {}])
```

client.fanpassLeaderboard

FanPass-specific leaderboards and statistics.

Source: `lib/src/fanpass_leaderboard.dart` · Class: `FanpassLeaderboardClient`.

`client.fanpassLeaderboard.getLeaderboard`

```
Future<FanpassLeaderboardResponse> getLeaderboard([FanpassLeaderboardOptions? options])
```

`client.fanpassLeaderboard.getUserComparison`

```
Future<FanpassUserComparisonResponse> getUserComparison(String userId, [JsonMap options = const
{}])
```

`client.fanpassLeaderboard.listCohorts`

```
Future<List<FanpassCohortStatus>> listCohorts([FanpassLeaderboardOptions? options])
```

`client.fanpassLeaderboard.getCohort`

```
Future<FanpassCohortStatus?> getCohort(String cohortSlug, [FanpassLeaderboardOptions? options])
```

client.notifications

Native SSE subscriptions and Web Push server registration APIs.

Source: `lib/src/notifications.dart` · Class: `NotificationsClient`.

`client.notifications.subscribe`

```
Unsubscribe subscribe(String userId, NotificationCallback callback, [SubscribeOptions options =
const SubscribeOptions()])
```

`client.notifications.getVapidKey`

```
Future<String?> getVapidKey()
```

`client.notifications.registerPush`

```
Future<JsonMap> registerPush(String userId, PushSubscriptionJSON subscription)
```

`client.notifications.unregisterPush`

```
Future<JsonMap> unregisterPush(String userId, String endpoint)
```

`client.notifications.requestPermissionAndSubscribe`

```
Future<PushSubscriptionJSON?> requestPermissionAndSubscribe(String userId, [String
serviceWorkerPath = '/sw.js'])
```

`client.notifications.generateServiceWorker`

```
String generateServiceWorker([JsonMap options = const {}])
```

`client.notifications.close` — native lifecycle

```
void close()
```

client.credentials

Credential types, issuance, user credentials and verification.

Source: `lib/src/credentials.dart` · Class: `CredentialsClient`.

`client.credentials.createType`

```
Future<CredentialType> createType(CreateCredentialTypeRequest request)
```

`client.credentials.listTypes`

```
Future<List<CredentialType>> listTypes([JsonMap? options])
```

`client.credentials.getType`

```
Future<CredentialType> getType(String typeId)
```

`client.credentials.updateType`

```
Future<CredentialType> updateType(String typeId, CreateCredentialTypeRequest request)
```

`client.credentials.activateType`

```
Future<CredentialType> activateType(String typeId)
```

`client.credentials.archiveType`

```
Future<CredentialType> archiveType(String typeId)
```

`client.credentials.issue`

```
Future<IssuedCredential> issue(IssueCredentialRequest request)
```

`client.credentials.listIssued`

```
Future<List<IssuedCredential>> listIssued([ListIssuedCredentialsOptions? options])
```

`client.credentials.revoke`

```
Future<IssuedCredential> revoke(String credentialId, [String? reason])
```

`client.credentials.suspend`

```
Future<IssuedCredential> suspend(String credentialId, [String? reason])
```

`client.credentials.reinstate`

```
Future<IssuedCredential> reinstate(String credentialId)
```

`client.credentials.optInUser`

```
Future<JsonMap> optInUser(String userExternalId)
```

`client.credentials.optOutUser`

```
Future<JsonMap> optOutUser(String userExternalId)
```

`client.credentials.getUserCredentials`

```
Future<UserCredentialsSummary> getUserCredentials(String userExternalId)
```

`client.credentials.verify`

```
Future<CredentialVerifyResult> verify(String verificationCode)
```

client.nft

Mint NFTs and list or retrieve mint records.

Source: `lib/src/nft.dart` · Class: `NftClient`.

`client.nft.mint`

```
Future<NFTMint> mint(MintNFTRequest request)
```

`client.nft.list`

```
Future<ListMintsResponse> list([ListMintsOptions? options])
```

`client.nft.get`

```
Future<NFTMint> get(String mintId)
```

client.partnerKeys

Privileged partner API-key administration.

Source: `lib/src/partner_keys.dart` · Class: `PartnerKeysClient`.

`client.partnerKeys.generate`

```
Future<OTTGenerateResponse> generate([OTTGenerateRequest? options])
```

`client.partnerKeys.redeem`

```
Future<OTTRedeemResponse> redeem(String ott)
```

`client.partnerKeys.updateOTTConfig`

```
Future<OTTConfigResponse> updateOTTConfig(String keyId, OTTConfigUpdate config)
```

`client.partnerKeys.getOTTConfig`

```
Future<OTTConfigResponse> getOTTConfig(String keyId)
```

`client.partnerKeys.requestOTT`

```
Future<OTTGenerateResponse> requestOTT(String keyId)
```

client.attestations

Attestation schemas, records, configuration and revocation.

Source: `lib/src/attestations.dart` · Class: `AttestationsClient`.

`client.attestations.listSchemas`

```
Future<List<AttestationSchema>> listSchemas([JsonMap? options])
```

`client.attestations.getSchema`

```
Future<AttestationSchema> getSchema(String schemaId)
```

`client.attestations.create`

```
Future<AttestationRecord> create(CreateAttestationRequest request)
```

`client.attestations.list`

```
Future<List<AttestationRecord>> list([ListAttestationsOptions? options])
```

`client.attestations.listBySubject`

```
Future<List<AttestationRecord>> listBySubject(String subjectAddress, [ListAttestationsOptions?
options])
```

`client.attestations.get`

```
Future<AttestationRecord> get(String attestationId)
```

`client.attestations.revoke`

```
Future<AttestationRecord> revoke(String attestationId, [String? reason])
```

`client.attestations.anchor`

```
Future<AttestationRecord> anchor(String attestationId)
```

`client.attestations.getConfig`

```
Future<AttestationConfig> getConfig()
```

`client.attestations.updateConfig`

```
Future<AttestationConfig> updateConfig(UpdateAttestationConfigRequest request)
```

`client.attestations.getWalletCredentials`

```
Future<List<WalletCredential>> getWalletCredentials(String walletAddress,
[ListWalletCredentialsOptions? options])
```

`client.attestations.getWalletCredential`

```
Future<WalletCredential> getWalletCredential(String walletAddress, String attestationUid)
```

`client.attestations.verifyWalletCredentials`

```
Future<WalletCredentialVerifySummary> verifyWalletCredentials(String walletAddress)
```

client.airkit

Create server-authorized AirKit sessions.

Source: `lib/src/airkit.dart` · Class: `AirKitClient`.

`client.airkit.createSession`

```
Future<AirKitSessionResponse> createSession()
```

Standalone ingestion clients

These clients are constructed separately; they are not `ProofChain` properties. Each request owns and closes its transport. Neither class has a public `close()` method.

IngestionClient

```
IngestionClient(IngestionClientOptions options)
```

`IngestionClient.ingest`

```
Future<IngestEventResponse> ingest(IngestEventRequest request)
```

`IngestionClient.ingestBatch`

```
Future<BatchIngestResponse> ingestBatch(BatchIngestRequest request)
```

`IngestionClient.getEventStatus`

```
Future<JsonMap> getEventStatus(String eventId)
```

EndUserIngestionClient

```
EndUserIngestionClient(EndUserIngestionClientOptions options)
```

`EndUserIngestionClient.setToken`

```
void setToken(String userToken)
```

`EndUserIngestionClient.ingest`

```
Future<IngestEventResponse> ingest(IngestEventRequest request)
```

`EndUserIngestionClient.ingestBatch`

```
Future<BatchIngestResponse> ingestBatch(BatchIngestRequest request)
```

Notification and upload support types

```
typedef NotificationCallback = void Function(NotificationEvent event);
typedef Unsubscribe = void Function();
```

`Unsubscribe` is a synchronous cancellation function, not `StreamSubscription`. Call it to stop connecting, active and scheduled reconnect work.

```
SubscribeOptions({this.onConnect, this.onDisconnect, this.onError, this.autoReconnect,
this.reconnectDelay})
```

```
UploadFile(List<int> bytes, {this.filename = 'document', this.contentType = 'application/octet-
stream'})
```

```
ProofChainOptions({this.apiKey, this.userToken, this.tenantId, this.baseUrl, this.tenant,
this.timeout, this.maxRetries})
```

17. Model and type catalog

All types below are exported by `package:proofchain_sdk/proofchain.dart`. The catalog is generated from `lib/src/models.dart`. It lists each named-constructor requirement and the number of optional constructor fields; use the source constructor or IDE completion for the full optional field types. Every generated class also supplies `Type.fromJson(JsonMap)` and `toJson()`. Required constructor fields describe the typed interface, not a promise that every server response contains that field; accessing a missing required getter can fail. Use raw `toJson()` when inspecting a response with unknown fields or a version mismatch.

The catalog intentionally distinguishes model construction from API authorization: constructing `Create*Request` does not grant permission to perform that operation. Do not pass an entire response model into a partial update without selecting the intended editable keys.

17.1 Generated model constructors

Type	Required named constructor fields	Optional
`AirKitSessionResponse`	`String auth_token`, `String partner_id`, `String build_env`, `num expires_in`	0
`AttestationSchema`	`String id`, `String name`, `String slug`, `String attestation_type`, `Map<String, Object?> schema_definition`, `bool revocable`, `num version`, `bool is_active`, `String created_at`	4
`AttestationRecord`	`String id`, `String uid`, `String tenant_id`, `String schema_id`, `String attester_address`, `String attester_type`, `String subject_address`, `Map<String, Object?> data`, `String status`, `String created_at`	11
`WalletCredential`	`String uid`, `String subject_address`, `Map<String, Object?> data`, `String status`, `String created_at`	7
`WalletCredentialVerifySummary`	`String wallet_address`, `num total_credentials`, `Map<String, num> credential_types`, `Map<String, Object?> latest_scores`, `bool is_verified`	0
`AttestationConfig`	`String id`, `String tenant_id`, `bool auto_attest_passports`, `bool auto_attest_cohort_scores`, `bool attest_on_score_change`, `num min_score_change_threshold`, `bool anchor_attestations`, `String reward_anchoring_mode`, `bool public_attestations`	0
`CreateAttestationRequest`	`String schema_id`, `String subject_address`, `Map<String, Object?> data`	4
`ListAttestationsOptions`	None	3
`ListWalletCredentialsOptions`	None	3

Type	Required named constructor fields	Optional
`UpdateAttestationConfigRequest`	None	6
`AttestationResult`	`String id`, `String ipfsHash`, `String documentHash`, `String gatewayUrl`, `String verifyUrl`, `String certificateId`, `EventStatus status`, `AttestationMode attestationMode`, `String timestamp`	2
`Event`	`String id`, `String eventType`, `String userId`, `String ipfsHash`, `String gatewayUrl`, `String certificateId`, `EventStatus status`, `AttestationMode attestationMode`, `String timestamp`	5
`Channel`	`String channelId`, `String name`, `ChannelState state`, `String createdAt`	0
`ChannelStatus`	`String channelId`, `String name`, `ChannelState state`, `num eventCount`, `num syncedCount`, `num pendingCount`, `String createdAt`	3
`Settlement`	`String channelId`, `String txHash`, `String merkleRoot`, `num eventCount`, `num blockNumber`, `num gasUsed`, `String settledAt`	0
`Certificate`	`String certificateId`, `String recipientName`, `String title`, `String ipfsHash`, `String verifyUrl`, `String qrCodeUrl`, `String issuedAt`, `bool revoked`	7
`Webhook`	`String id`, `String url`, `List<String> events`, `bool active`, `String createdAt`, `num failureCount`	1
`VerificationResult`	`bool valid`, `String ipfsHash`, `bool proofVerified`, `String message`	6
`SearchResult`	`num total`, `num page`, `num limit`, `List<Event> events`	0
`UsageStats`	`String periodStart`, `String periodEnd`, `num eventsCreated`, `num documentsAttested`, `num verifications`, `num apiCalls`, `num storageBytes`, `num channelsCreated`, `num settlements`	0
`TenantInfo`	`String tenantId`, `String name`, `String subdomain`, `String plan`, `String createdAt`, `num apiKeysCount`, `num eventsCount`, `num storageUsedBytes`	0
`StreamAck`	`num sequence`, `String channelId`, `bool received`	0
`AttestRequest`	`UploadFile file`, `String userId`	4
`CreateEventRequest`	`String eventType`, `String userId`	2
`ListEventsRequest`	None	7
`SearchRequest`	`String query`	6
`CreateChannelRequest`	`String name`	1

Type	Required named constructor fields	Optional
`StreamEventRequest`	`String eventType`, `String userId`	2
`IssueCertificateRequest`	`String recipientName`, `String title`	4
`ListCertificatesRequest`	None	3
`CreateWebhookRequest`	`String url`, `List<String> events`	1
`UpdateWebhookRequest`	None	3
`VaultFile`	`String id`, `String filename`, `String mimeType`, `num size`, `String ipfsHash`, `String userId`, `String accessMode`, `String createdAt`	2
`VaultFolder`	`String id`, `String name`, `String createdAt`	1
`VaultListResponse`	`List<VaultFile> files`, `List<VaultFolder> folders`, `num totalFiles`, `num totalSize`	0
`VaultStats`	`num totalFiles`, `num totalFolders`, `num totalSize`, `num usedQuota`, `num maxQuota`	0
`VaultUploadRequest`	`UploadFile file`, `String userId`	3
`SearchFilters`	None	9
`SearchQueryRequest`	None	4
`SearchResponse`	`num total`, `num offset`, `num limit`, `List<Event> results`	0
`PaginationMeta`	`num total`, `num page`, `num page_size`, `num total_pages`, `bool has_next`, `bool has_prev`	0
`UserEventHistoryResult`	`String id`, `String event_type`, `String event_source`, `String user_id`, `String status`, `String timestamp`, `bool has_blockchain_proof`, `String? blockchain_tx_hash`, `num? fanscore_v_e`, `num? fanscore_event_score`, `num? fanscore_contribution`, `String? fanscore_cohort`, `String? fanscore_subject`	0
`UserEventHistoryResponse`	`List<UserEventHistoryResult> results`, `num total`, `num offset`, `num limit`, `num query_time_ms`, `PaginationMeta? pagination`, `String? retention_warning`	0
`UserEventHistoryOptions`	None	3
`Facet`	`String key`, `num count`	0
`FacetsResponse`	`List<Facet> eventTypes`, `List<Facet> eventSources`, `List<Facet> statuses`, `List<Facet> users`	0
`SearchStats`	`num periodDays`, `num totalEvents`, `num uniqueUsers`, `num eventsPerDay`, `num totalStorageBytes`	0

Type	Required named constructor fields	Optional
`CertificateVerifyResult`	`String certificateId`, `String status`, `String recipientName`, `String title`, `String issuedAt`, `String ipfsHash`, `String verifyUrl`	4
`ProofVerifyResult`	`bool valid`, `String message`, `String verifiedAt`	0
`BatchVerifyResult`	`String batchId`, `String merkleRoot`, `num totalEvents`, `bool verified`	2
`EventBatchProof`	`String eventId`, `String certificateId`, `String batchId`, `num leafIndex`, `List<String> merkleProof`, `String merkleRoot`, `bool verified`	1
`ApiKey`	`String id`, `String name`, `String keyPrefix`, `List<String> permissions`, `String createdAt`, `bool isActive`	3
`BlockchainStats`	`num totalTransactions`, `num totalGasUsed`, `num totalEventsAttested`, `num pendingEvents`, `num chainId`, `String chainName`	2
`BlockchainProof`	`String certificateId`, `bool verified`, `String chainName`	5
`CreateApiKeyRequest`	`String name`	2
`Passport`	`String id`, `String tenant_id`, `String user_id`, `num level`, `num points`, `num experience`, `Map<String, Object?> traits`, `Map<String, Object?> custom_metadata`, `String created_at`, `String last_updated_at`	3
`PassportWithFields`	`super.id`, `super.tenant_id`, `super.user_id`, `super.level`, `super.points`, `super.experience`, `super.traits`, `super.custom_metadata`, `super.created_at`, `super.last_updated_at`, `List<FieldValue> field_values`	5
`FieldValue`	`String field_key`, `String field_name`, `Object? value`, `String data_type`, `bool manually_set`	1
`PassportTemplate`	`String id`, `String tenant_id`, `String name`, `String slug`, `bool is_default`, `bool is_active`, `String created_at`, `String updated_at`, `List<TemplateField> fields`	2
`TemplateField`	`String id`, `String template_id`, `String name`, `String field_key`, `String data_type`, `num sort_order`, `bool is_visible`, `bool is_computed`, `String created_at`	9
`Badge`	`String id`, `String tenant_id`, `String badge_id`, `String name`, `String rarity`, `Map<String, Object?> requirements`, `String created_at`	2

Type	Required named constructor fields	Optional
`Achievement`	`String id`, `String tenant_id`, `String achievement_id`, `String name`, `num points_reward`, `Map<String, Object?> requirements`, `String created_at`	2
`UserBadge`	`String id`, `String passport_id`, `String badge_id`, `Badge badge`, `String earned_at`	1
`UserAchievement`	`String id`, `String passport_id`, `String achievement_id`, `Achievement achievement`, `String earned_at`, `num progress`, `bool completed`	0
`PassportHistory`	`String id`, `String passport_id`, `String event_type`, `String created_at`	3
`CreatePassportRequest`	`String user_id`	6
`UpdatePassportRequest`	None	6
`CreateTemplateRequest`	`String name`, `String slug`	4
`CreateTemplateFieldRequest`	`String name`, `String field_key`	13
`CreateBadgeRequest`	`String badge_id`, `String name`	4
`CreateAchievementRequest`	`String achievement_id`, `String name`	4
`ListOptions`	None	2
`PassportV2Data`	`String user_id`, `PassportDefinition passport`, `Map<String, PassportFieldValue> data`, `List<Milestone> milestones`, `String generated_at`	0
`PassportFieldValue`	`String label`, `Object? value`	1
`PassportDefinition`	`String id`, `String tenant_id`, `String name`, `List<String> display_columns`, `Map<String, String> column_labels`, `Map<String, String> column_formats`, `bool is_default`, `bool is_active`, `bool is_public`, `String created_at`, `num threshold_count`	4
`Milestone`	`String id`, `String name`, `String milestone_type`, `num points_reward`, `String triggered_at`	4
`AvailableView`	`String id`, `String name`, `String view_name`, `bool is_builtin`, `List<ViewColumn> columns`	3
`ViewColumn`	`String name`, `String type`	1
`CreatePassportDefinitionRequest`	`String name`	8
`Wallet`	`String wallet_id`, `String address`, `String user_id`, `String wallet_type`, `String network`, `String status`, `String created_at`	5

Type	Required named constructor fields	Optional
`DualWallets`	`String user_id`, `Wallet asset_wallet`, `Wallet smart_wallet`, `String network`	0
`WalletBalance`	`String wallet_id`, `String address`, `String network`, `List<TokenBalance> balances`	0
`TokenBalance`	`String token`, `String symbol`, `String balance`, `num decimals`, `String source`	5
`NFT`	`String id`, `String wallet_id`, `String contract_address`, `String token_id`, `String network`, `String source`, `String acquired_at`	4
`SwapQuote`	`String from_token`, `String to_token`, `String from_amount`, `String to_amount`, `String min_to_amount`, `String exchange_rate`, `String quote_id`, `num slippage_bps`, `String network`, `bool liquidity_available`, `String expires_at`	0
`SwapResult`	`String transaction_hash`, `String status`, `String from_token`, `String to_token`, `String from_amount`, `String to_amount`, `String gas_used`	0
`WalletStats`	`num total_wallets`, `Map<String, num> by_type`, `Map<String, num> by_network`	0
`Transaction`	`String hash`, `String type`, `String from`, `String to`, `Object? value`, `String asset`, `String category`, `String block_num`, `String timestamp`	0
`TransactionHistory`	`String address`, `String network`, `num total_sent`, `num total_received`, `List<Transaction> transactions`	1
`ComprehensiveWalletInfo`	`String wallet_id`, `String address`, `String user_id`, `String wallet_type`, `String network`, `String status`, `bool supports_export`, `String created_at`	8
`UserWalletSummary`	`String user_id`, `num total_wallets`, `num total_nfts`, `num total_swaps`, `List<UserWalletSummaryWalletsItem> wallets`	0
`CreateWalletRequest`	`String user_id`	4
`CreateDualWalletsRequest`	`String user_id`	2
`SwapQuoteRequest`	`String from_token`, `String to_token`, `String from_amount`	2
`ExecuteSwapRequest`	`String wallet_id`, `String from_token`, `String to_token`, `String from_amount`	2
`AddNFTRequest`	`String contract_address`, `String token_id`, `String network`	2
`TransferRequest`	`String to_address`, `String amount`	5

Type	Required named constructor fields	Optional
`TransferResult`	`String tx_hash`, `String from`, `String to`, `String amount`, `String token`, `String network`, `String status`	1
`Token`	`String id`, `String contract_address`, `String network`, `String symbol`, `String name`, `num decimals`, `String token_standard`, `bool is_native_wrapper`, `bool is_verified`, `bool is_active`, `num display_order`, `bool is_global`	5
`CreateTokenRequest`	`String contract_address`, `String network`, `String symbol`, `String name`	8
`UpdateTokenRequest`	None	11
`WalletBalanceResponse`	`String address`, `String network`, `String native_balance`, `String native_symbol`, `List<TokenBalance> balances`	0
`UserWithWallets`	`String user_id`, `List<UserWithWalletsWalletsItem> wallets`	0
`UsersWithWalletsResponse`	`List<UserWithWallets> users`, `num total`, `num limit`, `num offset`	0
`EndUser`	`String id`, `String external_id`, `String status`, `num total_events`, `List<String> event_types`, `List<String> segments`, `Map<String, Object?> tags`, `num points_balance`, `num lifetime_points`, `Map<String, Object?> attributes`, `String created_at`	19
`EndUserListResponse`	`List<EndUser> users`, `num total`, `num page`, `num page_size`, `bool has_more`	0
`UserActivity`	`String event_id`, `String event_type`, `String timestamp`, `Map<String, Object?> data`	0
`UserActivityResponse`	`String user_id`, `List<UserActivity> activities`, `num total`, `num page`, `num page_size`	0
`CreateEndUserRequest`	`String external_id`	15
`UpdateEndUserRequest`	None	15
`ListEndUsersOptions`	None	9
`LinkWalletRequest`	None	4
`MergeUsersRequest`	`List<String> source_user_ids`, `String target_user_id`	0
`CreateUserWalletRequest`	None	2
`RegisterWalletRequest`	`String wallet_address`	1

Type	Required named constructor fields	Optional
`UserReward`	`String id`, `String reward_name`, `String reward_type`, `String status`	6
`UserRewardsResponse`	`String user_id`, `List<UserReward> rewards`, `num total`, `num page`, `num page_size`, `bool has_more`	0
`GDPRDeletionRequest`	`bool confirm`	3
`GDPRDeletionResponse`	`bool success`, `String user_id`, `String external_id`, `Map<String, num> deleted_records`	2
`GDPRPreviewResponse`	`Map<String, Object?> user`, `Map<String, num> would_delete`	1
`WalletCreationResult`	`bool success`, `String user_id`, `String wallet_address`, `String source`	2
`EnsureWalletResult`	`String user_id`, `EnsureWalletResultAssetWallet asset_wallet`, `EnsureWalletResultSmartWallet smart_wallet`, `String network`, `bool created`	0
`SetProfileRequest`	None	12
`RewardDefinition`	`String id`, `String name`, `String slug`, `String reward_type`, `num token_decimals`, `String token_chain`, `num nft_minted_pool_count`, `bool nft_is_soulbound`, `String trigger_type`, `num current_issued`, `bool is_active`, `bool attest_on_earn`, `String anchoring_mode`, `String created_at`	13
`EarnedReward`	`String id`, `String user_id`, `String user_external_id`, `String reward_name`, `String reward_type`, `String status`, `String earned_at`	5
`RewardUserReward`	`String id`, `String reward_name`, `String reward_type`, `String status`, `String earned_at`	11
`RewardAsset`	`String id`, `String definition_id`, `String asset_type`, `String file_path`, `String mime_type`, `num file_size`, `String created_at`	1
`CreateRewardDefinitionRequest`	`String name`, `String slug`	25
`ManualRewardRequest`	`String definition_id`, `List<String> user_ids`	2
`ClaimNFTResult`	`bool success`, `String reward_id`, `String tx_hash`, `String wallet_address`	1
`DistributeResult`	`bool success`, `String reward_id`, `String message`	0
`RevokeResult`	`bool success`, `String reward_id`, `String status`	0

Type	Required named constructor fields	Optional
`RewardAttestationResult`	`String attestation_id`, `String attestation_uid`, `String status`, `String subject_address`, `String reward_id`, `String reward_name`, `String earned_at`, `String created_at`	2
`AnchorResult`	`bool success`, `String reward_id`, `String attestation_id`, `String tx_hash`, `String status`	0
`RewardVerifyResult`	`bool is_valid`, `String attestation_uid`, `String status`, `String subject_address`, `String attester_address`, `String signature_type`, `bool on_chain`, `String reward_id`, `String reward_name`	2
`ListRewardsOptions`	None	4
`Quest`	`String id`, `String name`, `String slug`, `bool is_ordered`, `bool is_repeatable`, `List<String> prerequisite_quest_ids`, `String reward_mode`, `bool is_public`, `bool is_featured`, `num display_order`, `List<String> tags`, `String status`, `List<QuestStep> steps`, `num total_participants`, `num total_completions`, `String created_at`, `String updated_at`	35
`QuestCtaLink`	`String quest_id`, `String url`, `String resolver`, `String resolved_at`, `Map<String, Object?> meta`	0
`QuestCtaResolver`	`String key`, `String label`, `String description`, `bool configured`	0
`QuestStep`	`String id`, `String quest_id`, `String name`, `num order`, `String step_type`, `bool is_optional`	9
`UserQuestProgress`	`String id`, `String user_id`, `String quest_id`, `String quest_name`, `String status`, `num completion_count`, `num current_step_order`, `num steps_completed`, `num total_steps`, `num completion_percentage`, `List<StepProgress> step_progress`, `num points_earned`	4
`RewardEarned`	`String id`, `String reward_name`, `String reward_type`, `String status`, `String earned_at`	2
`StepProgress`	`String step_id`, `String step_name`, `num order`, `String status`	3
`StepStartResult`	`num step_index`, `String status`, `String quest_id`, `String quest_name`, `String step_name`	2
`StepCompletionResult`	`num step_index`, `bool step_completed`, `bool quest_completed`, `bool can_claim`, `num current_count`, `num target_count`, `num points_earned`	0
`ClaimRewardResult`	`String quest_id`, `String quest_name`, `String status`, `num points_awarded`, `String claimed_at`	1

Type	Required named constructor fields	Optional
`QuestWithProgress`	`Quest quest`	1
`CreateQuestRequest`	`String name`, `List<CreateQuestStepRequest> steps`	26
`CreateQuestStepRequest`	`String name`	12
`ListQuestsOptions`	None	6
`Schema`	`String id`, `String name`, `String version`, `String status`, `bool is_default`, `num usage_count`	4
`SchemaDetail`	`super.id`, `super.name`, `super.version`, `super.status`, `super.is_default`, `super.usage_count`, `Map<String, Object?> schema_definition`, `String yaml_content`	4
`SchemaField`	`String name`, `String type`	7
`ValidationResult`	`bool valid`, `String schema_name`, `List<SchemaValidationError> errors`	1
`SchemaValidationError`	`String field`, `String message`	1
`SchemaListResponse`	`List<Schema> schemas`, `num total`	0
`CreateSchemaRequest`	`String yaml_content`	0
`ValidateDataRequest`	`String schema_name`, `Map<String, Object?> data`	1
`ListSchemasOptions`	None	4
`DataViewColumn`	`String name`, `String type`	0
`DataViewInfo`	`String id`, `String name`, `String view_name`, `String description`, `bool is_builtin`, `List<DataViewColumn> columns`	0
`DataViewListResponse`	`List<DataViewSummary> own_views`, `List<DataViewSummary> public_views`, `List<DataViewSummary> builtin_views`	0
`DataViewSummary`	`String name`, `String description`	3
`DataViewDetail`	`String id`, `String name`, `String display_name`, `String description`, `String view_type`, `Object? computation`, `List<String> source_categories`, `bool is_public`, `String created_at`, `String updated_at`	0
`DataViewComputation`	`String type`	13
`TierDefinition`	`String name`, `num min`, `num max`	0
`DataViewExecuteResult`	`String view_name`, `String display_name`, `String identifier`, `String identifier_type`, `Map<String, Object?> data`, `String computed_at`, `num total_events`	0
`DataViewPreviewRequest`	`String wallet_address`, `Object? computation`	2

Type	Required named constructor fields	Optional
`DataViewPreviewResult`	`bool preview`, `String wallet_address`, `String computation_type`, `Map<String, Object?> result`, `num events_processed`, `num time_window_days`	0
`CreateDataViewRequest`	`String name`, `String display_name`, `String description`, `Object? computation`	3
`UpdateDataViewRequest`	None	5
`FanProfileView`	`String wallet_address`, `num fan_score`, `num total_events`, `Map<String, num> event_breakdown`, `String first_seen`, `String last_seen`, `String loyalty_tier`, `String computed_at`	0
`ActivitySummaryView`	`String wallet_address`, `num total_events`, `Map<String, num> event_count_by_type`, `String last_activity`, `num active_days`, `num period_days`, `String computed_at`	0
`EventMetadata`	`List<EventMetadataEventTypesItem> event_types`, `num total_events`, `List<String> categories`	0
`ViewTemplate`	`String id`, `String name`, `String description`, `DataViewComputation computation`	0
`LeaderboardUserProfile`	`String external_id`, `String? display_name`, `String? first_name`, `String? last_name`, `String? email`, `String? avatar_url`, `String? country`, `String? city`, `Map<String, Object?> attributes`	0
`CohortLeaderboardEntry`	`num rank`, `String user_id`, `num score`, `num percentile_global`, `num? percentile_filtered`	2
`CohortGroupStats`	`num? filtered_avg_percentile`, `num? filtered_top_n_avg_percentile`, `num global_count`, `num global_avg_percentile`	0
`CohortLeaderboardResponse`	`String cohort_id`, `String cohort_name`, `Map<String, Object?> filter`, `num total_users`, `CohortGroupStats group_stats`, `List<CohortLeaderboardEntry> leaderboard`, `CohortLeaderboardEntry? current_user`, `bool current_user_in_leaderboard`	0
`CohortDefinition`	`String id`, `String tenant_id`, `String name`, `String slug`, `String scoring_type`, `String status`, `String created_at`, `String updated_at`	5
`UserCohortBreakdownEntry`	`String cohort_id`, `String cohort_slug`, `String cohort_name`, `num user_percentile`, `num? filtered_group_avg_percentile`, `num global_group_avg_percentile`	2
`UserBreakdownResponse`	`String user_id`, `Map<String, Object?> filter`, `List<UserCohortBreakdownEntry> cohorts`	0
`CohortLeaderboardOptions`	None	6

Type	Required named constructor fields	Optional
`ListCohortsOptions`	None	3
`FanpassLeaderboardUserProfile`	`String external_id`, `String? display_name`, `String? first_name`, `String? last_name`, `String? email`, `String? avatar_url`, `String? country`, `String? city`, `Map<String, Object?> attributes`	0
`FanpassLeaderboardEntry`	`num rank`, `String user_id`, `num fan_score`, `num normalized_score`, `num raw_score`, `num percentile`	2
`FanpassGroupStats`	`num avg_fan_score`, `num top_n_avg_fan_score`, `num global_count`, `num global_avg_fan_score`	0
`FanpassCohortStatus`	`String cohort_slug`, `String cohort_name`, `num users_scored`, `String status`	0
`FanpassLeaderboardResponse`	`String? aggregation_rule_id`, `String? aggregation_rule_name`, `Map<String, Object?> filter`, `num total_users`, `FanpassGroupStats group_stats`, `List<FanpassLeaderboardEntry> leaderboard`, `List<FanpassCohortStatus> cohorts`, `FanpassLeaderboardEntry? current_user`, `bool current_user_in_leaderboard`	0
`FanpassUserComparisonResponse`	`String user_id`, `Map<String, Object?> filter`, `List<UserCohortBreakdownEntry> cohorts`	0
`FanpassLeaderboardOptions`	None	7
`PushSubscriptionJSON`	`String endpoint`, `PushSubscriptionJSONKeys keys`	0
`NotificationEvent`	`String id`, `NotificationEventType event`, `Map<String, Object?> data`, `String tenant_id`, `String user_id`, `String timestamp`	0
`CredentialType`	`String id`, `String name`, `String slug`, `num display_order`, `Map<String, Object?> schema_definition`, `bool is_revocable`, `num max_active_per_user`, `bool auto_renew`, `String default_visibility`, `bool requires_opt_in`, `String status`, `num total_issued`, `num total_active`, `String created_at`	5
`IssuedCredential`	`String id`, `String credential_type_id`, `String credential_type_name`, `String credential_type_slug`, `String user_id`, `String verification_code`, `Map<String, Object?> credential_data`, `String visibility`, `String status`, `bool is_valid`, `String issued_at`, `num verification_count`	5
`CredentialVerifyResult`	`bool valid`, `String status`, `String credential_type`, `String credential_type_slug`, `Map<String, Object?> credential_data`, `String issued_at`, `String issuer`, `num verification_count`	4

Type	Required named constructor fields	Optional
`UserCredentialsSummary`	`bool credentials_enabled`, `num total_credentials`, `num active_credentials`, `List<IssuedCredential> credentials`	0
`CreateCredentialTypeRequest`	`String name`	15
`IssueCredentialRequest`	`String credential_type_id`, `String user_id`	4
`ListIssuedCredentialsOptions`	None	5
`MintNFTRequest`	`String recipientAddress`, `String name`	10
`NFTMint`	`String id`, `String tenant_id`, `String recipient_address`, `String? recipient_user_id`, `num? token_id`, `String? tx_hash`, `String contract_address`, `String network`, `String? content_hash`, `num cert_type`, `num? validity_days`, `Map<String, Object?>? metadata`, `String? metadata_uri`, `String image_url`, `String? image_storage_path`, `String name`, `String? description`, `String status`, `String? error_message`, `String? gas_used`, `String? created_at`, `String? updated_at`	0
`ListMintsOptions`	None	5
`ListMintsResponse`	`List<NFTMint> mints`, `num total`	0
`OTTGenerateRequest`	None	2
`OTTGenerateResponse`	`String ott`, `num expires_in`	0
`OTTRedeemResponse`	`String user_id`, `String tenant_id`, `String? tenant_name`, `Map<String, Object?> session_data`, `String issued_at`, `String expires_at`	0
`OTTConfigUpdate`	None	5
`OTTConfigResponse`	`String partner_key_id`, `String partner_name`, `bool ott_enabled`, `num ott_ttl_seconds`, `String ott_redemption_mode`, `num ott_jwt_ttl_seconds`	1
`IngestEventRequest`	`String userId`, `String eventType`	5
`IngestEventResponse`	`String eventId`, `String certificateId`, `String status`	2
`BatchIngestRequest`	`List<IngestEventRequest> events`	0
`BatchIngestResponse`	`num totalEvents`, `num queued`, `num failed`, `List<IngestEventResponse> results`	0
`IngestionClientOptions`	`String apiKey`	2
`EndUserIngestionClientOptions`	`String userToken`, `String tenantId`	2

Type	Required named constructor fields	Optional
`ComprehensiveWalletInfoBalances`	`String address`, `String network`, `List<TokenBalance> tokens`, `String fetched_at`	0
`ComprehensiveWalletInfoNfts`	`num total`, `List<NFT> items`	0
`ComprehensiveWalletInfoActivity`	`List<SwapResult> recent_swaps`, `num total_swaps`	0
`UserWalletSummaryWalletsItem`	`String wallet_id`, `String address`, `String wallet_type`, `String network`, `bool supports_export`, `String created_at`	4
`UserWithWalletsWalletsItem`	`String wallet_id`, `String address`, `String wallet_type`, `String network`, `String status`	1
`EnsureWalletResultAssetWallet`	`String wallet_id`, `String address`, `String wallet_type`, `String network`	0
`EnsureWalletResultSmartWallet`	`String wallet_id`, `String address`, `String wallet_type`, `String network`	0
`EventMetadataEventTypesItem`	`String event_type`, `num count`, `String last_seen`	0
`PushSubscriptionJSONKeys`	`String p256dh`, `String auth`	0
`UserWalletSummaryWalletsItemBalances`	`String address`, `String network`, `List<TokenBalance> tokens`, `String fetched_at`	0

17.2 String unions and aliases

Dart string aliases do not enforce the JavaScript union members at runtime. Check the JavaScript type definition and server contract for accepted literal values.

```
typedef AttestationMode = String;
```

```
typedef EventStatus = String;
```

```
typedef ChannelState = String;
```

```
typedef QuestCtaType = String;
```

```
typedef NotificationEventType = String;
```

```
typedef OTTRequestResponse = OTTGenerateResponse;
```

17.3 Handwritten support and error types

`JsonModel`, `JsonMap`, `UploadFile`, `ProofChainOptions`, `SubscribeOptions`, `NotificationCallback`, `Unsubscribe`, `ProofChainError`, `AuthenticationError`, `AuthorizationError`, `NotFoundError`, `ValidationError`, `ValidationErrorDetail`, `RateLimitError`, `ServerError`, `NetworkError`, and `TimeoutError` are exported support types. Error semantics and arbitrary validation detail handling are covered in chapter 5. `HttpClient` and JSON conversion helpers are internal implementation details, not exported transport-extension APIs.